Final Project

Smart Campus Facility Booking System - Zayed University

Shahd Ali Alhosani - 202409559

Maryam Omar Alshehhi - 202411158

Zayed University

ICS220 - 20606 Programming Fundamentals

Section 101

Prof. Areej Abdulfattah

Friday, 8 May 2026

Google Colab Link:
https://colab.research.google.com/drive/1bb_oQ9DtfkFQfY7Qbqy_ysXnp05bChwf?usp=sharing

Github repository Link:
https://github.com/shahdalhosani/Final_Assignment_Shahd_Alhosani_202409559_Maryam_Alshehhi_202411158

1. UML Class Diagram

In this section, we present the UML class diagram for the Smart Campus Facility Booking System. The diagram shows the main classes in the system, including User, Admin, Facility, Booking, and BookingSystem, along with their attributes and methods. We also showed the relationships between the classes, including inheritance, and association along with the multiplicity. This design follows a modular structure, where each class has a specific role, making the system organized, clear, and easy to manage.

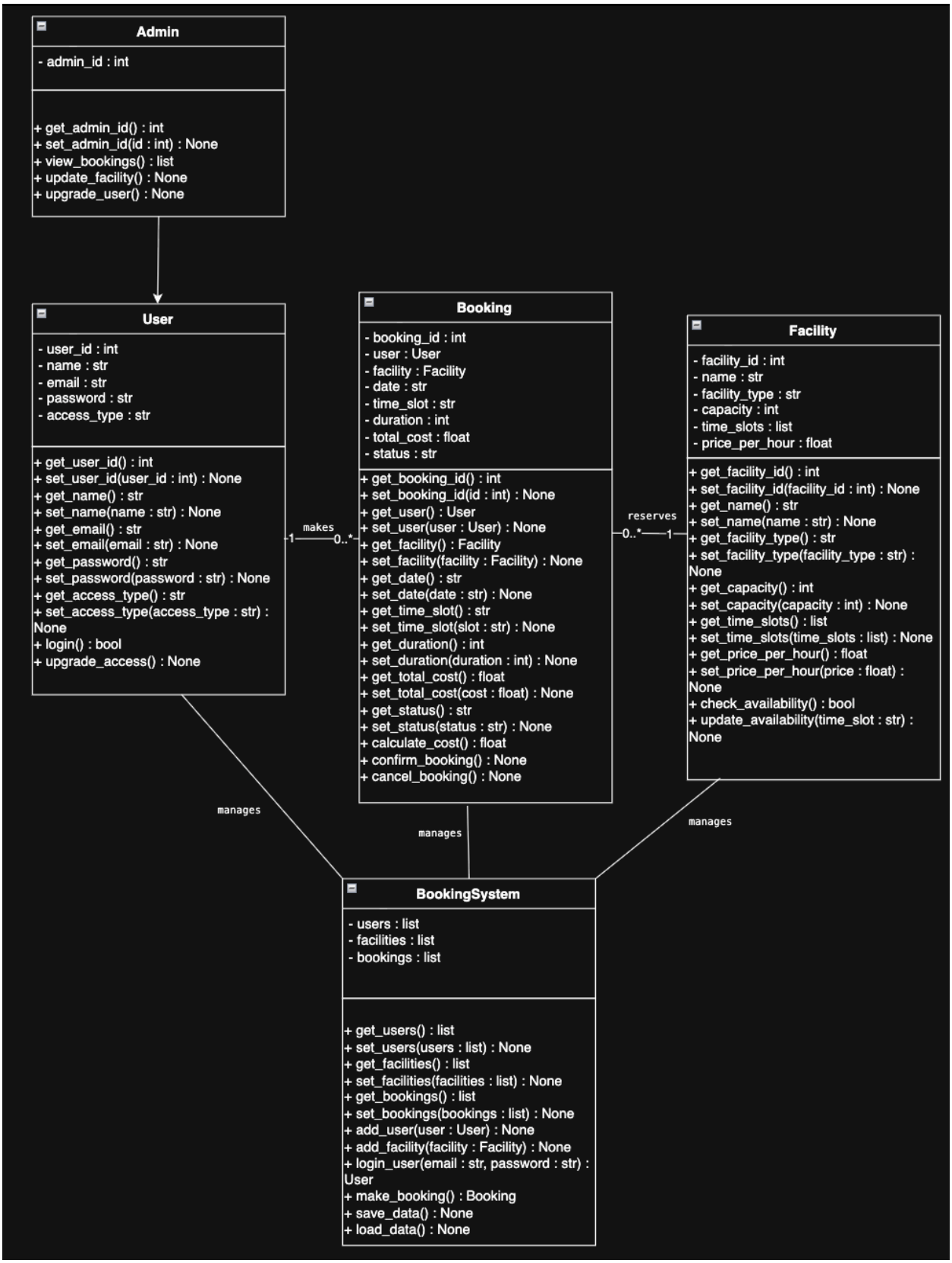

Admin
- admin_id : int
+ get_admin_id() : int
+ set_admin_id(id : int) : None
+ view_bookings() : list
+ update_facility() : None
+ upgrade_user() : None
User
- user_id : int
- name : str
- email : str
- password : str
- access_type : str
+ get_user_id() : int
+ set_user_id(user_id : int) : None
+ get_name() : str
+ set_name(name : str) : None
+ get_email() : str
+ set_email(email : str) : None
+ get_password() : str
+ set_password(password : str) : None
+ get_access_type() : str
+ set_access_type(access_type : str) : None
+ login() : bool
+ upgrade_access() : None
Booking
- booking_id : int
- user : User
- facility : Facility
- date : str
- time_slot : str
- duration : int
- total_cost : float
- status : str
+ get_booking_id() : int
+ set_booking_id(id : int) : None
+ get_user() : User
+ set_user(user : User) : None
+ get_facility() : Facility
+ set_facility(facility : Facility) : None
+ get_date() : str
+ set_date(date : str) : None
+ get_time_slot() : str
+ set_time_slot(slot : str) : None
+ get_duration() : int
+ set_duration(duration : int) : None
+ get_total_cost() : float
+ set_total_cost(cost : float) : None
+ get_status() : str
+ set_status(status : str) : None
+ calculate_cost() : float
+ confirm_booking() : None
+ cancel_booking() : None
Facility
- facility_id : int
- name : str
- facility_type : str
- capacity : int
- time_slots : list
- price_per_hour : float
+ get_facility_id() : int
+ set_facility_id(facility_id : int) : None
+ get_name() : str
+ set_name(name : str) : None
+ get_facility_type() : str
+ set_facility_type(facility_type : str) : None
+ get_capacity() : int
+ set_capacity(capacity : int) : None
+ get_time_slots() : list
+ set_time_slots(time_slots : list) : None
+ get_price_per_hour() : float
+ set_price_per_hour(price : float) : None
+ check_availability() : bool
+ update_availability(time_slot : str) : None
BookingSystem
- users : list
- facilities : list
- bookings : list
+ get_users() : list
+ set_users(users : list) : None
+ get_facilities() : list
+ set_facilities(facilities : list) : None
+ get_bookings() : list
+ set_bookings(bookings : list) : None
+ add_user(user : User) : None
+ add_facility(facility : Facility) : None
+ login_user(email : str, password : str) : User
+ make_booking() : Booking
+ save_data() : None
+ load_data() : None
makes
1
0..*
reserves
0..*
1
manages
manages
manages

Diagram link: https://drive.google.com/file/d/1AmbVpgb8_--d0-a7qDl8orQaObLKRNDt/view?usp=sharing

Description - Identify Classes and Relationships

In this section, we explained the relationships between each class, such as inheritance between Admin and User, and associations between User and Booking, and Booking and Facility. Additionally, the BookingSystem class is associated with User, Booking, and Facility. We also explained what each relationship mean within our system.

User

The User class represents any person who uses the system, such as a student or a staff member. In this class, we included important details like user ID, name, email, password, and access type (for example, standard or premium). These details help identify the user and allow them to log in. We also included methods such as login() to allow the user to access the system and upgrade_access() to change their access level. In addition, getter and setter methods are included so that the system can safely read and update user information when needed. The User class is connected to the Booking class, where one user can make zero or many bookings (0..*). This means a user might not have any bookings yet, or they can have multiple bookings. The User class is also connected to the BookingSystem class, which manages all users in the system.

Admin

The Admin class represents a special type of user who has more control over the system. In this class, we included an admin ID and methods such as view_bookings(), update_facility(), and upgrade_user(). These methods allow the admin to monitor and manage the system. The Admin class is connected to the User class through inheritance, which means Admin inherits from User and has all the basic features of a normal user with extra permissions. Because of this, the admin can use all the features of the User class in addition to their own.

Facility

The Facility class represents the places or resources that can be booked, such as study rooms, lecture halls, or sports courts. In this class, we included details like facility ID, name, facility type, capacity, time slots, and price per hour. These details help users choose the right facility and understand its availability and cost. We also added methods like check_availability() to see if the facility is free and update_availability() to change its schedule. Getter and setter methods are included to manage facility data. The Facility class is connected to the Booking class, where one facility can be linked to many bookings (0..*). This means the same facility can be booked multiple times by different users at different times. The Facility is also connected to the BookingSystem class, which manages all facilities.

Booking

The Booking class represents a reservation made by a user. This class connects the User and Facility classes, meaning each booking is made by one user and is for one facility. In this class, we included attributes such as booking ID, user, facility, date, time slot, duration, total cost, and status. The user and facility attributes store references to the User and Facility objects, which

helps connect the booking to the correct person and the correct facility. These details help track and manage each reservation. We also added methods like calculate_cost() to calculate the total price based on time and facility, confirm_booking() to confirm the reservation, and cancel_booking() to cancel it if needed. Getter and setter methods are included to manage booking data. The receipt details are also handled inside this class through total_cost, status, and booking confirmation. The Booking class is connected to the User class (many bookings for one user) and to the Facility class (many bookings for one facility). It is also connected to the BookingSystem class, which stores and manages all bookings.

BookingSystem

The BookingSystem class represents the main system that controls everything. It brings all parts together. In this class, we included lists of users, facilities, and bookings. These lists help store all the data in one place. We also included methods such as add_user() to add new users, add_facility() to add new facilities, login_user() to allow users to log in, make_booking() to create new bookings, and save_data() and load_data() to store and retrieve data using files. Getter and setter methods are also included to manage these lists. The BookingSystem class is connected to the User, Booking, and Facility classes, which shows that it manages all parts of the system.

Class 1	Relationship	Class 2	Description
Admin	Inheritance (is a)	User	Admin is a special type of User with extra permissions.
User	Association (makes)	Booking	One user can make zero or many bookings (1 -> 0..*).
Booking	Association (reserves)	Facility	Each booking is for one facility, while a facility can have many bookings (0..* -> 1).
BookingSystem	Association (manages)	User	BookingSystem manages all users.
BookingSystem	Association (manages)	Booking	BookingSystem manages all bookings.
BookingSystem	Association (manages)	Facility	BookingSystem manages all facilities.

2. Python Implementation

In the Python implementation, we created a Smart Campus Facility Booking System using classes and a GUI. We used classes such as User, Admin, Facility, Booking, and BookingSystem

to organize the code clearly. The system allows users to create an account, log in, view facilities, make bookings, modify or delete bookings, and upgrade their access. It also allows the admin to view all users and bookings, upgrade users, update facility time slots, and check facility usage. We also used pickle to save and load data, so the information is not lost when the program closes.

Import Libraries

```
#Pickle is used to save and load data
import pickle

#Tkinter is used to create the GUI
import tkinter as tk
```

User Class

```
#This class stores user information
class User:
    def __init__(self, user_id, name, email, password,
access_type="Standard"):
        self.user_id = user_id
        self.name = name
        self.email = email
        self.password = password
        self.access_type = access_type

    #Getter and setter for user ID
    def get_user_id(self):
        return self.user_id

    def set_user_id(self, user_id):
        self.user_id = user_id

    #Getter and setter for name
    def get_name(self):
        return self.name

    def set_name(self, name):
        self.name = name

    #Getter and setter for email
    def get_email(self):
        return self.email

    def set_email(self, email):
        self.email = email

    #Getter and setter for password
    def get_password(self):
        return self.password
```

```
    def set_password(self, password):
        self.password = password

    #Getter and setter for access type
    def get_access_type(self):
        return self.access_type

    def set_access_type(self, access_type):
        self.access_type = access_type

    #Check if email and password are correct
    def login(self, email, password):
        return self.email == email and self.password == password

    #Upgrade user from Standard to Premium
    def upgrade_access(self):
        self.access_type = "Premium"
```

Admin Class

```
#Admin is a special type of User
class Admin(User):
    def __init__(self, user_id, name, email, password, access_type,
admin_id):
        super().__init__(user_id, name, email, password, access_type)
        self.admin_id = admin_id

    #Getter and setter for admin ID
    def get_admin_id(self):
        return self.admin_id

    def set_admin_id(self, admin_id):
        self.admin_id = admin_id

    #Admin can view all bookings
    def view_bookings(self, bookings):
        return bookings

    #Admin can update facility time slots
    def update_facility(self, facility, time_slots):
        facility.set_time_slots(time_slots)

    #Admin can upgrade a user
    def upgrade_user(self, user):
        user.upgrade_access()
```

Facility Class

```
#This class stores facility details
class Facility:
```

```
    def __init__(self, facility_id, name, facility_type, capacity,
time_slots, price_per_hour):
        self.facility_id = facility_id
        self.name = name
        self.facility_type = facility_type
        self.capacity = capacity
        self.time_slots = time_slots
        self.price_per_hour = price_per_hour

    #Getter and setter for facility ID
    def get_facility_id(self):
        return self.facility_id

    def set_facility_id(self, facility_id):
        self.facility_id = facility_id

    #Getter and setter for facility name
    def get_name(self):
        return self.name

    def set_name(self, name):
        self.name = name

    #Getter and setter for facility type
    def get_facility_type(self):
        return self.facility_type

    def set_facility_type(self, facility_type):
        self.facility_type = facility_type

    #Getter and setter for capacity
    def get_capacity(self):
        return self.capacity

    def set_capacity(self, capacity):
        self.capacity = capacity

    #Getter and setter for time slots
    def get_time_slots(self):
        return self.time_slots

    def set_time_slots(self, time_slots):
        self.time_slots = time_slots

    #Getter and setter for price per hour
    def get_price_per_hour(self):
        return self.price_per_hour

    def set_price_per_hour(self, price_per_hour):
        self.price_per_hour = price_per_hour
```

```
    #Check if the selected time slot is available
    def check_availability(self, time_slot):
        return time_slot in self.time_slots

    #Update availability by removing the booked time slot
    def update_availability(self, time_slot):
        if time_slot in self.time_slots:
            self.time_slots.remove(time_slot)
```

Booking Class

```
#This class stores booking details
class Booking:
    def __init__(self, booking_id, user, facility, date, time_slot,
duration, total_cost=0, status="Pending"):
        self.booking_id = booking_id
        self.user = user
        self.facility = facility
        self.date = date
        self.time_slot = time_slot
        self.duration = duration
        self.total_cost = total_cost
        self.status = status

    #Getter and setter for booking ID
    def get_booking_id(self):
        return self.booking_id

    def set_booking_id(self, booking_id):
        self.booking_id = booking_id

    #Getter and setter for user
    def get_user(self):
        return self.user

    def set_user(self, user):
        self.user = user

    #Getter and setter for facility
    def get_facility(self):
        return self.facility

    def set_facility(self, facility):
        self.facility = facility

    #Getter and setter for date
    def get_date(self):
        return self.date
```

```
    def set_date(self, date):
        self.date = date

    #Getter and setter for time slot
    def get_time_slot(self):
        return self.time_slot

    def set_time_slot(self, time_slot):
        self.time_slot = time_slot

    #Getter and setter for duration
    def get_duration(self):
        return self.duration

    def set_duration(self, duration):
        self.duration = duration

    #Getter and setter for total cost
    def get_total_cost(self):
        return self.total_cost

    def set_total_cost(self, total_cost):
        self.total_cost = total_cost

    #Getter and setter for status
    def get_status(self):
        return self.status

    def set_status(self, status):
        self.status = status

    #Calculate total cost
    def calculate_cost(self):
        self.total_cost = self.duration *
self.facility.get_price_per_hour()
        return self.total_cost

    #Confirm booking
    def confirm_booking(self):
        self.status = "Confirmed"

    #Cancel booking
    def cancel_booking(self):
        self.status = "Cancelled"
```

Booking System Class

```
#This class controls the whole system
#It stores all users, facilities, and bookings in one place
class BookingSystem:
```

```
    #Create empty lists for users, facilities, and bookings
    def __init__(self):
        self.users = []
        self.facilities = []
        self.bookings = []

    #Getter and setter for users
    def get_users(self):
        return self.users

    def set_users(self, users):
        self.users = users

    #Getter and setter for facilities
    def get_facilities(self):
        return self.facilities

    def set_facilities(self, facilities):
        self.facilities = facilities

    #Getter and setter for bookings
    def get_bookings(self):
        return self.bookings

    def set_bookings(self, bookings):
        self.bookings = bookings

    #Add user to the system
    def add_user(self, user):
        self.users.append(user)

    #Add facility to the system
    def add_facility(self, facility):
        self.facilities.append(facility)

    #Login user by checking email and password
    def login_user(self, email, password):
        for user in self.users:
            if user.login(email, password):
                return user

    #Make booking and return Booking if available
    def make_booking(self, booking):
        facility = booking.get_facility()
        time_slot = booking.get_time_slot()

        #If the time slot is available, the booking can continue
        if facility.check_availability(time_slot):
            booking.calculate_cost()
            booking.confirm_booking()
```

```
            facility.update_availability(time_slot)
            self.bookings.append(booking)
            return booking

    #Save data using pickle
    #This saves the whole system object into a file
    def save_data(self):
        with open("campus_booking_system.pkl", "wb") as file:
            pickle.dump(self, file)

    #Load data using pickle
    #This loads the old saved data if the file exists
    def load_data(self):
        try:
            with open("campus_booking_system.pkl", "rb") as file:
                saved_system = pickle.load(file)
                self.users = saved_system.users
                self.facilities = saved_system.facilities
                self.bookings = saved_system.bookings
        except FileNotFoundError:
            pass
```

Create System Object and Add Starting Data

```
#Create the system object
system = BookingSystem()

#Load old saved data if it exists
system.load_data()

#Add sample facilities only if there is no saved data
#This avoids adding the same facilities many times
if len(system.get_facilities()) == 0:
    system.add_facility(Facility(1, "Study Room C", "Study Room", 4,
["9-11 AM", "2-4 PM"], 0))
    system.add_facility(Facility(2, "Tennis Court", "Sports Court", 4,
["3-5 PM", "5-7 PM"], 15))
    system.add_facility(Facility(3, "Conference Hall", "Event Hall",
60, ["10-12 PM", "1-3 PM"], 40))

#Add sample users only if there is no saved data
if len(system.get_users()) == 0:
    system.add_user(User(1, "Student One", "student1@zu.ac.ae",
"1111"))
    system.add_user(Admin(2, "Admin", "admin@zu.ac.ae", "admin",
"Admin", 200))

#Save the data
#This saves the starting data into the pickle file
system.save_data()
```

Main Menu Page

```
#Clear the window before opening a new page
def clear_window():
    for widget in window.winfo_children():
        widget.destroy()

#Show main menu
def main_menu():
    clear_window()

    #Main title of the system
    tk.Label(window, text="Smart Campus Facility Booking System -
Zayed University", font=("Times New Roman", 16, "bold")).pack()

    #Buttons that take the user to different pages
    tk.Button(window, text="Login", width=30,
command=login_page).pack()
    tk.Button(window, text="Create Account", width=30,
command=create_account_page).pack()
    tk.Button(window, text="View Facilities", width=30,
command=view_facilities_without_user).pack()
    tk.Button(window, text="Save Data", width=30,
command=save_data).pack()
    tk.Button(window, text="Exit", width=30,
command=window.destroy).pack()
```

Login Page

```
#Show login page
#This page allows users and admins to log in
def login_page():
    clear_window()

    #Page title
    tk.Label(window, text="Login", font=("Times New Roman", 16,
"bold")).pack()

    #Email input
    tk.Label(window, text="Email").pack()
    email_entry = tk.Entry(window, width=35)
    email_entry.pack()

    #Password input
    #Show="*" hides the password while typing
    tk.Label(window, text="Password").pack()
    password_entry = tk.Entry(window, width=35, show="*")
    password_entry.pack()

    #This label shows messages like errors or success
    result_label = tk.Label(window, text="")
```

```
    result_label.pack()

    #Check login details
    def login_action():
        email = email_entry.get()
        password = password_entry.get()

        #Make sure the user filled both fields
        if email == "" or password == "":
            result_label.config(text="Please enter email and
password.")
            return

        #Check if the email and password match a saved user
        user = system.login_user(email, password)

        #If the user exists, open their dashboard
        if user:
            user_dashboard(user)

        #If not, show an error message
        else:
            result_label.config(text="Wrong email or password.")

    #Login button runs login_action
    tk.Button(window, text="Login", width=25,
command=login_action).pack()

    #Back button returns to main menu
    tk.Button(window, text="Back", width=25, command=main_menu).pack()
```

Create Account Page

```
#Show create account page
#This allows a new student/user to create an account
def create_account_page():
    clear_window()

    #Page title
    tk.Label(window, text="Create Account", font=("Times New Roman",
16, "bold")).pack()

    #Name input
    tk.Label(window, text="Name").pack()
    name_entry = tk.Entry(window, width=35)
    name_entry.pack()

    #Email input
    tk.Label(window, text="Email").pack()
    email_entry = tk.Entry(window, width=35)
```

```
    email_entry.pack()

    #Password input
    tk.Label(window, text="Password").pack()
    password_entry = tk.Entry(window, width=35, show="*")
    password_entry.pack()

    #Message label
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Create a new user account
    def create_action():
        name = name_entry.get()
        email = email_entry.get()
        password = password_entry.get()

        #Make sure all fields are filled
        if name == "" or email == "" or password == "":
            result_label.config(text="Please fill all fields.")
            return

        #Check if the email already exists
        for user in system.get_users():
            if user.get_email() == email:
                result_label.config(text="This email already exists.")
                return

        #Create a new user ID based on the number of users
        user_id = len(system.get_users()) + 1

        #Create a new User object
        new_user = User(user_id, name, email, password)

        #Add the new user to the system
        system.add_user(new_user)

        #Save the new user data
        system.save_data()

        #Show success message
        result_label.config(text="Account created successfully.")

    #Button to create account
    tk.Button(window, text="Create Account", width=25,
command=create_action).pack()

    #Button to go back
    tk.Button(window, text="Back", width=25, command=main_menu).pack()
```

User Dashboard Page

```
#Show user dashboard after login
def user_dashboard(user):
    clear_window()

    #Dashboard title and user details
    tk.Label(window, text="User Dashboard", font=("Times New Roman",
16, "bold")).pack()
    tk.Label(window, text="Welcome, " + user.get_name()).pack()
    tk.Label(window, text="Access Type: " +
user.get_access_type()).pack()

#Functions to send the current user to the next page when a button is
clicked

    #Open the facilities page
    def open_facilities():
        view_facilities(user)

    #Open the booking page
    def open_booking():
        booking_page(user)

    #Open the user's own bookings
    def open_my_bookings():
        view_my_bookings(user)

    #Open the page to modify user details
    def open_modify_details():
        modify_user_details_page(user)

    #Delete the current user account
    def open_delete_account():
        delete_user_account(user)

    #Upgrade the current user to Premium
    def open_upgrade():
        upgrade_current_user(user)

    #Open admin dashboard if user is admin
    def open_admin():
        admin_dashboard(user)

    #User buttons
    tk.Button(window, text="View Facilities", width=30,
command=open_facilities).pack()
    tk.Button(window, text="Make Booking", width=30,
command=open_booking).pack()
    tk.Button(window, text="View My Bookings", width=30,
command=open_my_bookings).pack()
```

```
    tk.Button(window, text="Modify My Details", width=30,
command=open_modify_details).pack()
    tk.Button(window, text="Delete My Account", width=30,
command=open_delete_account).pack()
    tk.Button(window, text="Upgrade to Premium", width=30,
command=open_upgrade).pack()

    #If the user is admin, show admin button
    if user.get_access_type() == "Admin":
        tk.Button(window, text="Admin Dashboard", width=30,
command=open_admin).pack()

    #Logout returns to main menu
    tk.Button(window, text="Logout", width=30,
command=main_menu).pack()
```

View Facilities Pages and Find Facility

```
#View facilities when no user is logged in
#This allows visitors to see available facilities before logging in
def view_facilities_without_user():
    clear_window()

    #Page title
    tk.Label(window, text="Facilities", font=("Times New Roman", 16,
"bold")).pack()

    #Text box to display all facility details
    text_box = tk.Text(window, width=75, height=18)
    text_box.pack()

    #Loop through all facilities and print their details
    for facility in system.get_facilities():
        text_box.insert(tk.END, "Facility ID: " +
str(facility.get_facility_id()) + "\n")
        text_box.insert(tk.END, "Name: " + facility.get_name() + "\n")
        text_box.insert(tk.END, "Type: " +
facility.get_facility_type() + "\n")
        text_box.insert(tk.END, "Capacity: " +
str(facility.get_capacity()) + "\n")
        text_box.insert(tk.END, "Available Slots: " +
str(facility.get_time_slots()) + "\n")
        text_box.insert(tk.END, "Fee: " +
str(facility.get_price_per_hour()) + " AED/hour\n")
        text_box.insert(tk.END, "------------------------------\n")

    #Back button
    tk.Button(window, text="Back", width=25, command=main_menu).pack()

#View facilities when user is logged in
```

```
def view_facilities(user):
    clear_window()

    #Page title
    tk.Label(window, text="Facilities", font=("Times New Roman", 16,
"bold")).pack()

    #Text box for facility information
    text_box = tk.Text(window, width=75, height=18)
    text_box.pack()

    #Show each facility in the text box
    for facility in system.get_facilities():
        text_box.insert(tk.END, "Facility ID: " +
str(facility.get_facility_id()) + "\n")
        text_box.insert(tk.END, "Name: " + facility.get_name() + "\n")
        text_box.insert(tk.END, "Type: " +
facility.get_facility_type() + "\n")
        text_box.insert(tk.END, "Capacity: " +
str(facility.get_capacity()) + "\n")
        text_box.insert(tk.END, "Available Slots: " +
str(facility.get_time_slots()) + "\n")
        text_box.insert(tk.END, "Fee: " +
str(facility.get_price_per_hour()) + " AED/hour\n")
        text_box.insert(tk.END, "------------------------------\n")

    #Return to the same user's dashboard
    def back_to_dashboard():
        user_dashboard(user)

    #Back button
    tk.Button(window, text="Back", width=25,
command=back_to_dashboard).pack()

#Find facility by ID
#This searches for a facility using the facility ID entered by the
user
def find_facility(facility_id):
    for facility in system.get_facilities():
        if facility.get_facility_id() == facility_id:
            return facility
```

Check User Booking Permission

```
#Check Standard/Premium permission
#Premium users can book different facility types
#Standard users can only book one facility type
def check_user_permission(user, facility):
    if user.get_access_type() == "Premium":
        return True
```

```
    #Check the user's old bookings
    for booking in system.get_bookings():
        if booking.get_user().get_email() == user.get_email():
            old_type = booking.get_facility().get_facility_type()
            new_type = facility.get_facility_type()

            #If the new facility type is different, do not allow it
            if old_type != new_type:
                return False
    #If there is no problem, allow the booking
    return True
```

Make Booking Page

```
#Show booking page
def booking_page(user):
    clear_window()

    #Page title
    tk.Label(window, text="Make Booking", font=("Times New Roman", 16,
"bold")).pack()

    #Facility ID input
    tk.Label(window, text="Facility ID").pack()
    facility_entry = tk.Entry(window, width=35)
    facility_entry.pack()

    #Date input
    tk.Label(window, text="Date").pack()
    date_entry = tk.Entry(window, width=35)
    date_entry.pack()

    #Time slot input
    tk.Label(window, text="Time Slot").pack()
    time_entry = tk.Entry(window, width=35)
    time_entry.pack()

    #Duration input
    tk.Label(window, text="Duration in hours").pack()
    duration_entry = tk.Entry(window, width=35)
    duration_entry.pack()

    #Label for messages and cost preview
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Show total cost before booking
    def preview_cost():
        try:
```

```
            facility_id = int(facility_entry.get())
            duration = int(duration_entry.get())

            #Find the selected facility
            facility = find_facility(facility_id)

            #If facility exists, calculate cost
            if facility:
                cost = duration * facility.get_price_per_hour()
                result_label.config(text="Total cost will be " +
str(cost) + " AED.")

            #If the ID is wrong, show error
            else:
                result_label.config(text="Facility not found.")

        #This handles wrong number input
        except ValueError:
            result_label.config(text="Please enter numbers
correctly.")

    #Confirm booking
    def confirm_booking():
        try:
            facility_id = int(facility_entry.get())
            date = date_entry.get()
            time_slot = time_entry.get()
            duration = int(duration_entry.get())

            #Make sure date and time are not empty
            if date == "" or time_slot == "":
                result_label.config(text="Please fill all fields.")
                return

            #Find the facility
            facility = find_facility(facility_id)

            #Continue only if facility exists
            if facility:
                #Check if the user is allowed to book this facility
type
                if not check_user_permission(user, facility):
                    result_label.config(text="Standard users can book
only one facility type.")
                    return

                #Create a new booking ID
                booking_id = len(system.get_bookings()) + 1

                #Create a booking object
```

```
                booking = Booking(booking_id, user, facility, date,
time_slot, duration)

                #Try to make the booking in the system
                result = system.make_booking(booking)

                #If booking is successful, save the data
                if result:
                    system.save_data()
                    result_label.config(text="Booking confirmed.
Booking ID: " + str(result.get_booking_id()))

                #If time slot is not available, show message
                else:
                    result_label.config(text="This time slot is not
available.")

            #If facility is not found, show message
            else:
                result_label.config(text="Facility not found.")

        #This handles wrong number input
        except ValueError:
            result_label.config(text="Please enter correct numbers.")

    #Back to user dashboard
    def back_to_dashboard():
        user_dashboard(user)

    #Buttons for booking page
    tk.Button(window, text="Show Cost Before Confirmation", width=35,
command=preview_cost).pack()
    tk.Button(window, text="Confirm Booking", width=35,
command=confirm_booking).pack()
    tk.Button(window, text="Back", width=35,
command=back_to_dashboard).pack()
```

View Bookings for the Logged-in User Page

```
#Show bookings for the logged-in user
def view_my_bookings(user):
    clear_window()

    #Page title
    tk.Label(window, text="My Bookings", font=("Times New Roman", 16,
"bold")).pack()

    #Text box to display bookings
    text_box = tk.Text(window, width=75, height=16)
    text_box.pack()
```

```
    #This checks if the user has bookings or not
    found = False

    #Go through all bookings in the system
    for booking in system.get_bookings():

        #Show only the bookings that belong to this user
        if booking.get_user().get_email() == user.get_email():
            found = True
            text_box.insert(tk.END, "Booking ID: " +
str(booking.get_booking_id()) + "\n")
            text_box.insert(tk.END, "Facility: " +
booking.get_facility().get_name() + "\n")
            text_box.insert(tk.END, "Date: " + booking.get_date() + "\
n")
            text_box.insert(tk.END, "Time: " + booking.get_time_slot()
+ "\n")
            text_box.insert(tk.END, "Cost: " +
str(booking.get_total_cost()) + " AED\n")
            text_box.insert(tk.END, "Status: " + booking.get_status()
+ "\n")
            text_box.insert(tk.END, "-----------------------------\
n")

     #If the user has no bookings, show this message
    if not found:
        text_box.insert(tk.END, "You have no bookings yet.")

    #Open modify booking page
    def open_modify_booking():
        modify_booking_page(user)

    #Open delete booking page
    def open_delete_booking():
        delete_booking_page(user)

    #Back to dashboard
    def back_to_dashboard():
        user_dashboard(user)

    #Buttons for booking actions
    tk.Button(window, text="Modify Booking Date", width=30,
command=open_modify_booking).pack()
    tk.Button(window, text="Delete Booking", width=30,
command=open_delete_booking).pack()
    tk.Button(window, text="Back", width=30,
command=back_to_dashboard).pack()
```

Modify Booking Page

```
#Modify booking date page
def modify_booking_page(user):
    clear_window()

    #Page title
    tk.Label(window, text="Modify Booking", font=("Times New Roman",
16, "bold")).pack()

    #Booking ID input
    tk.Label(window, text="Booking ID").pack()
    booking_id_entry = tk.Entry(window, width=35)
    booking_id_entry.pack()

    #New date input
    tk.Label(window, text="New Date").pack()
    new_date_entry = tk.Entry(window, width=35)
    new_date_entry.pack()

    #Message label
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Save the new booking date
    def save_changes():
        try:
            booking_id = int(booking_id_entry.get())
            new_date = new_date_entry.get()

            #Make sure the new date is entered
            if new_date == "":
                result_label.config(text="Please enter new date.")
                return

            #Search for the booking that belongs to this user
            for booking in system.get_bookings():
                if booking.get_booking_id() == booking_id and
booking.get_user().get_email() == user.get_email():

                    #Update the date
                    booking.set_date(new_date)

                    #Save the updated data
                    system.save_data()

                    #Show success message
                    result_label.config(text="Booking date updated.")
                    return

            #If booking was not found
```

```
            result_label.config(text="Booking not found.")
        #If booking ID is not a number
        except ValueError:
            result_label.config(text="Please enter a correct booking
ID.")

    #Back to my bookings page
    def back_to_bookings():
        view_my_bookings(user)

    #Buttons
    tk.Button(window, text="Save Changes", width=25,
command=save_changes).pack()
    tk.Button(window, text="Back", width=25,
command=back_to_bookings).pack()
```

Delete Booking Page

```
#Delete booking page
#The user can delete one of their bookings
def delete_booking_page(user):
    clear_window()

    #Page title
    tk.Label(window, text="Delete Booking", font=("Times New Roman",
16, "bold")).pack()

    #Booking ID input
    tk.Label(window, text="Booking ID").pack()
    booking_id_entry = tk.Entry(window, width=35)
    booking_id_entry.pack()

    #Message label
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Delete booking action
    def delete_booking_action():
        try:
            booking_id = int(booking_id_entry.get())

            #Search for the booking using booking ID and user email
            for booking in system.get_bookings():
                if booking.get_booking_id() == booking_id and
booking.get_user().get_email() == user.get_email():

                    #Mark the booking as cancelled
                    booking.cancel_booking()

                    #Add the time slot back to the facility because it
```

```
is free again

booking.get_facility().get_time_slots().append(booking.get_time_slot()
)

                    #Remove the booking from the system
                    system.get_bookings().remove(booking)

                    #Save the updated data
                    system.save_data()

                    #Show success message
                    result_label.config(text="Booking deleted.")
                    return

            #If booking was not found
            result_label.config(text="Booking not found.")

        #If booking ID is not a number
        except ValueError:
            result_label.config(text="Please enter a correct booking
ID.")

    #Back to my bookings page
    def back_to_bookings():
        view_my_bookings(user)

    #Buttons
    tk.Button(window, text="Delete", width=25,
command=delete_booking_action).pack()
    tk.Button(window, text="Back", width=25,
command=back_to_bookings).pack()
```

Modify User Details Page

```
#Modify user details page
#This allows the user to update their name and email
def modify_user_details_page(user):
    clear_window()

    #Page title
    tk.Label(window, text="Modify My Details", font=("Times New
Roman", 16, "bold")).pack()

    #New name input
    tk.Label(window, text="New Name").pack()
    name_entry = tk.Entry(window, width=35)
    name_entry.pack()

    #New email input
```

```
    tk.Label(window, text="New Email").pack()
    email_entry = tk.Entry(window, width=35)
    email_entry.pack()

    #Message label
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Save the new user details
    def save_details():
        new_name = name_entry.get()
        new_email = email_entry.get()

        #Make sure both fields are filled
        if new_name == "" or new_email == "":
            result_label.config(text="Please fill all fields.")
            return

        #Update user name and email
        user.set_name(new_name)
        user.set_email(new_email)

        #Save changes
        system.save_data()

        #Show success message
        result_label.config(text="User details updated.")

    #Back to dashboard
    def back_to_dashboard():
        user_dashboard(user)

    #Buttons
    tk.Button(window, text="Save", width=25,
command=save_details).pack()
    tk.Button(window, text="Back", width=25,
command=back_to_dashboard).pack()

#Delete user account
#This removes the current user from the users list
def delete_user_account(user):
    system.get_users().remove(user)
    system.save_data()
    main_menu()

#Upgrade user access
#This upgrades the current user to Premium
def upgrade_current_user(user):
    user.upgrade_access()
```

```
    system.save_data()
    user_dashboard(user)
```

Admin Dashboard Page

```
#Admin dashboard
def admin_dashboard(admin):
    clear_window()

    #Page title
    tk.Label(window, text="Admin Dashboard", font=("Times New Roman",
16, "bold")).pack()

    #Open all bookings page
    def open_all_bookings():
        view_all_bookings(admin)

    #Open all users page
    def open_all_users():
        view_all_users(admin)

    #Open upgrade user page
    def open_upgrade_user():
        admin_upgrade_user_page(admin)

    #Open update facility page
    def open_update_facility():
        admin_update_facility_page(admin)

    #Open facility usage page
    def open_usage():
        view_facility_usage(admin)

    #Back to normal user dashboard
    def back_to_dashboard():
        user_dashboard(admin)

    #Admin buttons
    tk.Button(window, text="View All Bookings", width=35,
command=open_all_bookings).pack()
    tk.Button(window, text="View All Users", width=35,
command=open_all_users).pack()
    tk.Button(window, text="Upgrade User Access", width=35,
command=open_upgrade_user).pack()
    tk.Button(window, text="Update Facility Availability", width=35,
command=open_update_facility).pack()
    tk.Button(window, text="View Facility Usage", width=35,
command=open_usage).pack()
    tk.Button(window, text="Back", width=35,
command=back_to_dashboard).pack()
```

Admin View Bookings and Users

```
#Admin can view all bookings
def view_all_bookings(admin):
    clear_window()

    #Page title
    tk.Label(window, text="All Bookings", font=("Times New Roman", 16,
"bold")).pack()

    #Text box for all booking details
    text_box = tk.Text(window, width=75, height=18)
    text_box.pack()

    #If there are no bookings, show a message
    if len(system.get_bookings()) == 0:
        text_box.insert(tk.END, "No bookings yet.")

    #Otherwise, show all bookings
    else:
        for booking in system.get_bookings():
            text_box.insert(tk.END, "Booking ID: " +
str(booking.get_booking_id()) + "\n")
            text_box.insert(tk.END, "User: " +
booking.get_user().get_name() + "\n")
            text_box.insert(tk.END, "Facility: " +
booking.get_facility().get_name() + "\n")
            text_box.insert(tk.END, "Date: " + booking.get_date() + "\
n")
            text_box.insert(tk.END, "Time: " + booking.get_time_slot()
+ "\n")
            text_box.insert(tk.END, "Cost: " +
str(booking.get_total_cost()) + " AED\n")
            text_box.insert(tk.END, "Status: " + booking.get_status()
+ "\n")
            text_box.insert(tk.END, "-----------------------------\
n")

    #Back to admin dashboard
    def back_to_admin():
        admin_dashboard(admin)

    #Back button
    tk.Button(window, text="Back", width=25,
command=back_to_admin).pack()


#Admin can view all users
#This shows all users registered in the system
def view_all_users(admin):
    clear_window()
```

```
    #Page title
    tk.Label(window, text="All Users", font=("Times New Roman", 16,
"bold")).pack()

    #Text box for user details
    text_box = tk.Text(window, width=75, height=18)
    text_box.pack()

    #Loop through all users and show their details
    for user in system.get_users():
        text_box.insert(tk.END, "User ID: " + str(user.get_user_id())
+ "\n")
        text_box.insert(tk.END, "Name: " + user.get_name() + "\n")
        text_box.insert(tk.END, "Email: " + user.get_email() + "\n")
        text_box.insert(tk.END, "Access Type: " +
user.get_access_type() + "\n")
        text_box.insert(tk.END, "------------------------------\n")

    #Back to admin dashboard
    def back_to_admin():
        admin_dashboard(admin)

    #Back button
    tk.Button(window, text="Back", width=25,
command=back_to_admin).pack()
```

Admin Upgrade User Access

```
#Admin upgrades user page
#Admin enters the user ID and upgrades that user to Premium
def admin_upgrade_user_page(admin):
    clear_window()

    #Page title
    tk.Label(window, text="Upgrade User", font=("Times New Roman", 16,
"bold")).pack()

    #User ID input
    tk.Label(window, text="User ID").pack()
    user_id_entry = tk.Entry(window, width=35)
    user_id_entry.pack()

    #Message label
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Upgrade user action
    def upgrade_user_action():
        try:
```

```
            user_id = int(user_id_entry.get())

            #Search for the user by ID
            for user in system.get_users():
                if user.get_user_id() == user_id:

                    #Admin upgrades the user
                    admin.upgrade_user(user)

                    #Save the change
                    system.save_data()

                    #Show success message
                    result_label.config(text="User upgraded to
Premium.")
                    return

            #If user ID is not found
            result_label.config(text="User not found.")

        #If user ID is not a number
        except ValueError:
            result_label.config(text="Please enter a correct user
ID.")

    #Back to admin dashboard
    def back_to_admin():
        admin_dashboard(admin)

    #Buttons
    tk.Button(window, text="Upgrade", width=25,
command=upgrade_user_action).pack()
    tk.Button(window, text="Back", width=25,
command=back_to_admin).pack()
```

Admin Update Facility Availability

```
#Admin updates facility page
#Admin can add a new available time slot to a facility
def admin_update_facility_page(admin):
    clear_window()

    #Page title
    tk.Label(window, text="Update Facility", font=("Times New Roman",
16, "bold")).pack()

    #Facility ID input
    tk.Label(window, text="Facility ID").pack()
    facility_id_entry = tk.Entry(window, width=35)
    facility_id_entry.pack()
```

```
    #New time slot input
    tk.Label(window, text="New Time Slot").pack()
    new_slot_entry = tk.Entry(window, width=35)
    new_slot_entry.pack()

    #Message label
    result_label = tk.Label(window, text="")
    result_label.pack()

    #Update facility action
    def update_facility_action():
        try:
            facility_id = int(facility_id_entry.get())
            new_slot = new_slot_entry.get()

            #Make sure the new time slot is entered
            if new_slot == "":
                result_label.config(text="Please enter new time
slot.")
                return

            #Find the facility by ID
            facility = find_facility(facility_id)

            #If facility exists, add the new slot
            if facility:
                slots = facility.get_time_slots()
                slots.append(new_slot)
                admin.update_facility(facility, slots)
                system.save_data()
                result_label.config(text="Facility availability
updated.")

            #If facility does not exist
            else:
                result_label.config(text="Facility not found.")
         #If facility ID is not a number
        except ValueError:
            result_label.config(text="Please enter a correct facility
ID.")

    #Back to admin dashboard
    def back_to_admin():
        admin_dashboard(admin)

    #Buttons
    tk.Button(window, text="Update", width=25,
command=update_facility_action).pack()
```

```
    tk.Button(window, text="Back", width=25,
command=back_to_admin).pack()
```

Admin View Facility Usage

```
#Admin can view facility usage
#This shows how many bookings each facility has

def view_facility_usage(admin):
    clear_window()

    #Page title
    tk.Label(window, text="Facility Usage and Capacity", font=("Times
New Roman", 16, "bold")).pack()

    #Text box for facility usage
    text_box = tk.Text(window, width=75, height=18)
    text_box.pack()

    #Loop through each facility
    for facility in system.get_facilities():
        #Count how many bookings are made for this facility
        count = 0

        #Check each booking
        for booking in system.get_bookings():
            if booking.get_facility().get_facility_id() ==
facility.get_facility_id():
                count = count + 1

        #Show facility usage details
        text_box.insert(tk.END, "Facility: " + facility.get_name() +
"\n")
        text_box.insert(tk.END, "Capacity: " +
str(facility.get_capacity()) + "\n")
        text_box.insert(tk.END, "Number of Bookings: " + str(count) +
"\n")
        text_box.insert(tk.END, "Available Slots: " +
str(facility.get_time_slots()) + "\n")
        text_box.insert(tk.END, "------------------------------\n")

    #Back to admin dashboard
    def back_to_admin():
        admin_dashboard(admin)

    #Back button
    tk.Button(window, text="Back", width=25,
command=back_to_admin).pack()

#Save data manually
```

```
#This function is used by the Save Data button in the main menu
def save_data():
    system.save_data()
```

Create and Run Main Window

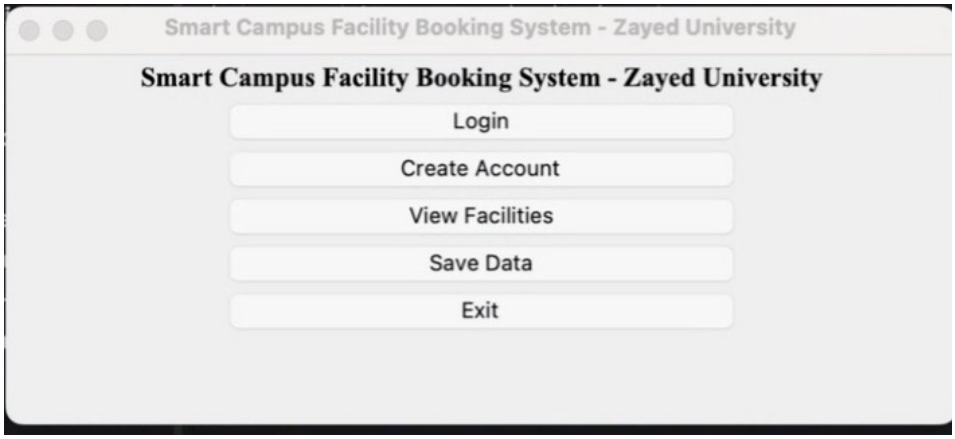
```
#Create the main window
window = tk.Tk()

#Set title
window.title("Smart Campus Facility Booking System - Zayed
University")

#Set size
window.geometry("750x650")

#Start with main menu
main_menu()

#Keep window open
window.mainloop()
```

Full Code Link:
https://colab.research.google.com/drive/1_mknjzmH7wEeRzcNgWFFCbnuGtt7xtZv?usp=sharing

The code is stored in the code folder. Each file contains a different part of the Smart Campus Facility Booking System, such as the classes, booking system, GUI pages, and main program. This makes the code easier to read, understand, and manage. We also included the .pkl file because it saves the system data, such as users, facilities, and bookings, so the information is not lost when the program closes.

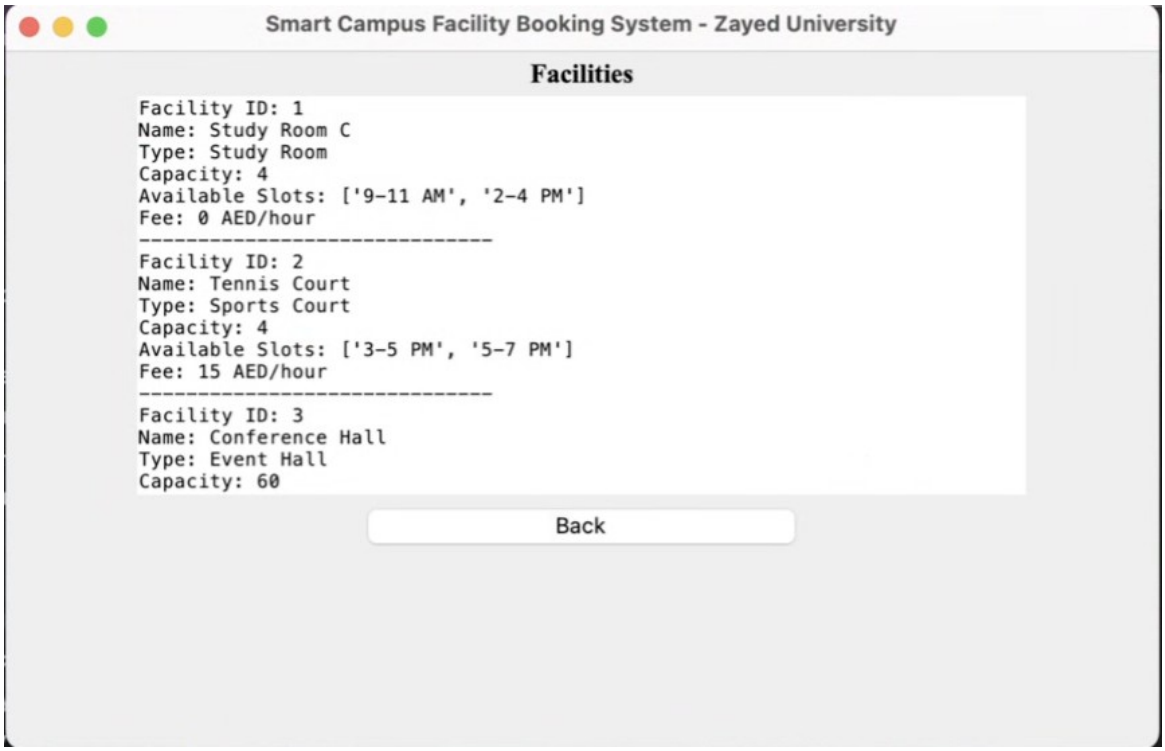
3. Testcases

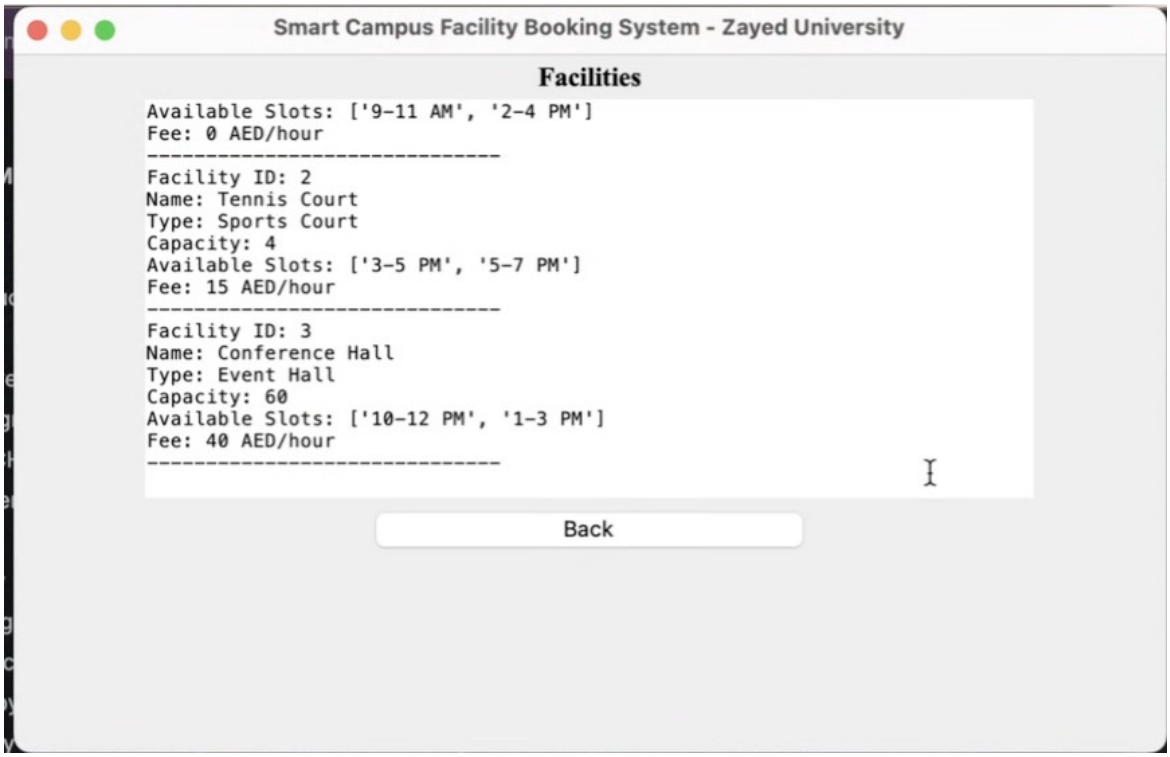
1 - Main Page Display



When we opened the system, the main page appeared with all the main options like Login, Create Account, View Facilities, Save Data, and Exit. This test checks that the system starts correctly and shows the main menu with all the required options for the user to choose from.

2 - View Facilities

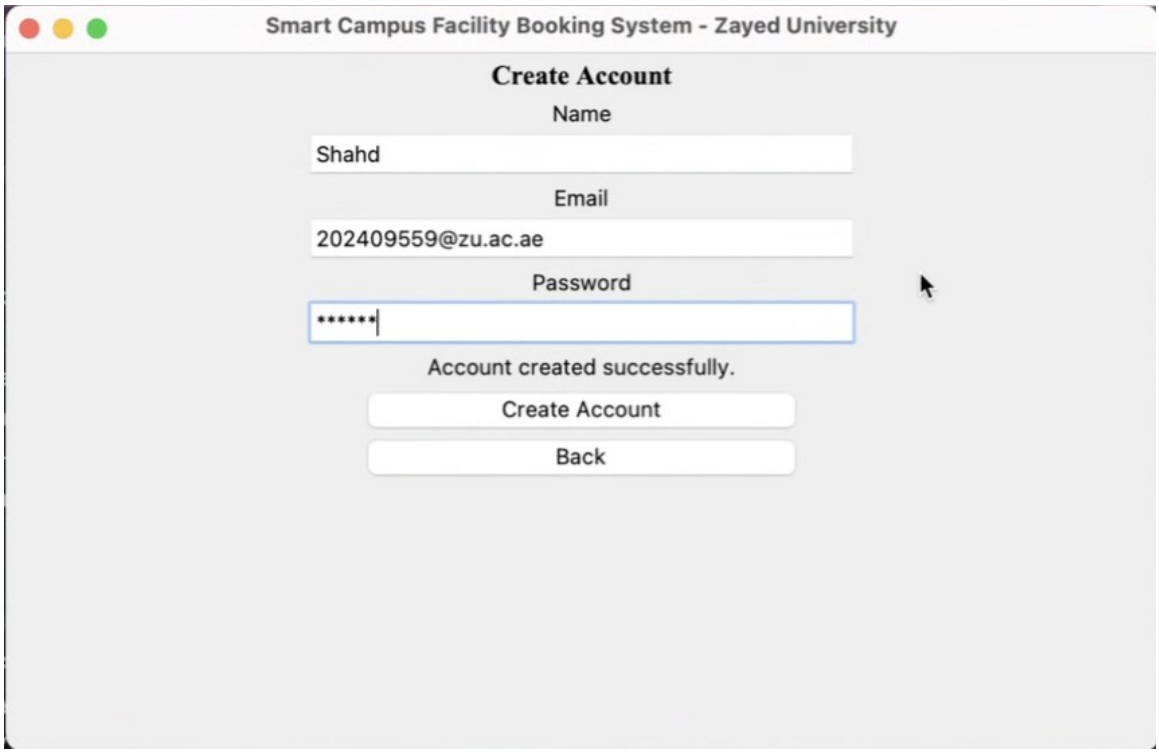




When we clicked on View Facilities from the main page, the system displayed all available facilities such as Study Room, Tennis Court, and Conference Hall. It also showed details like capacity, available time slots, and fees for each facility. This test checks that the system correctly

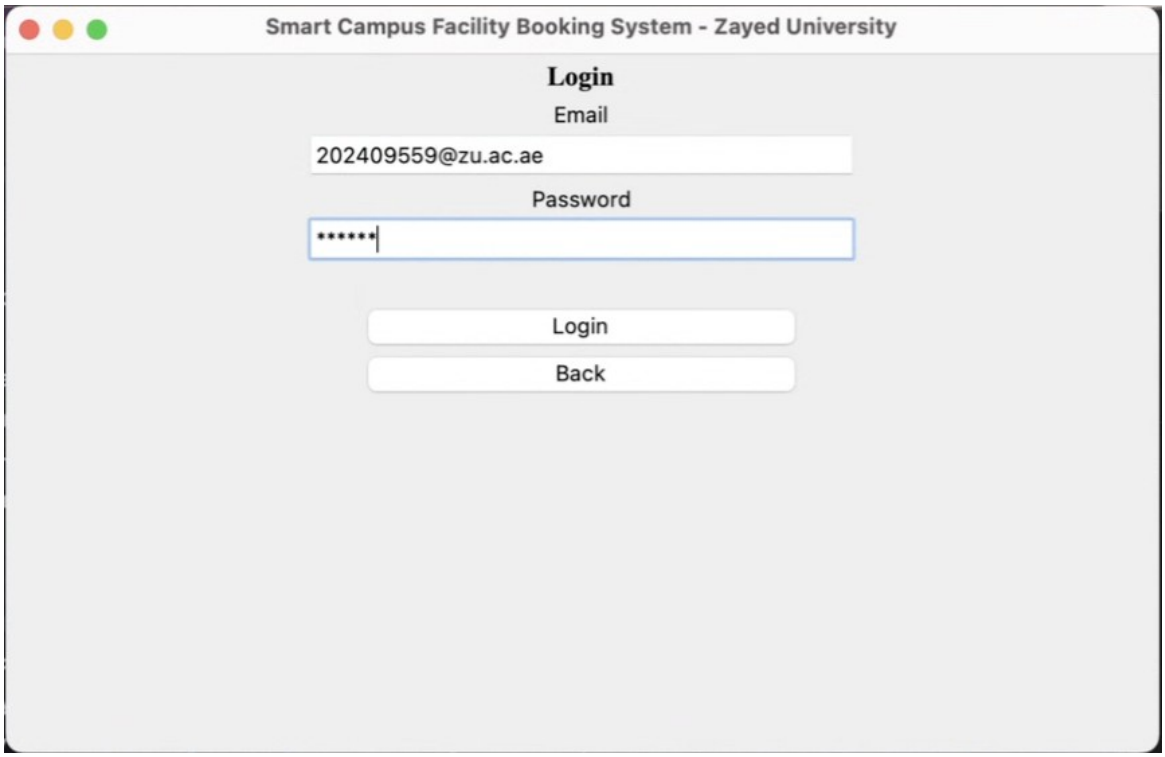
displays all facilities and their information so the user can understand the options before making a booking.

3 - Create Account

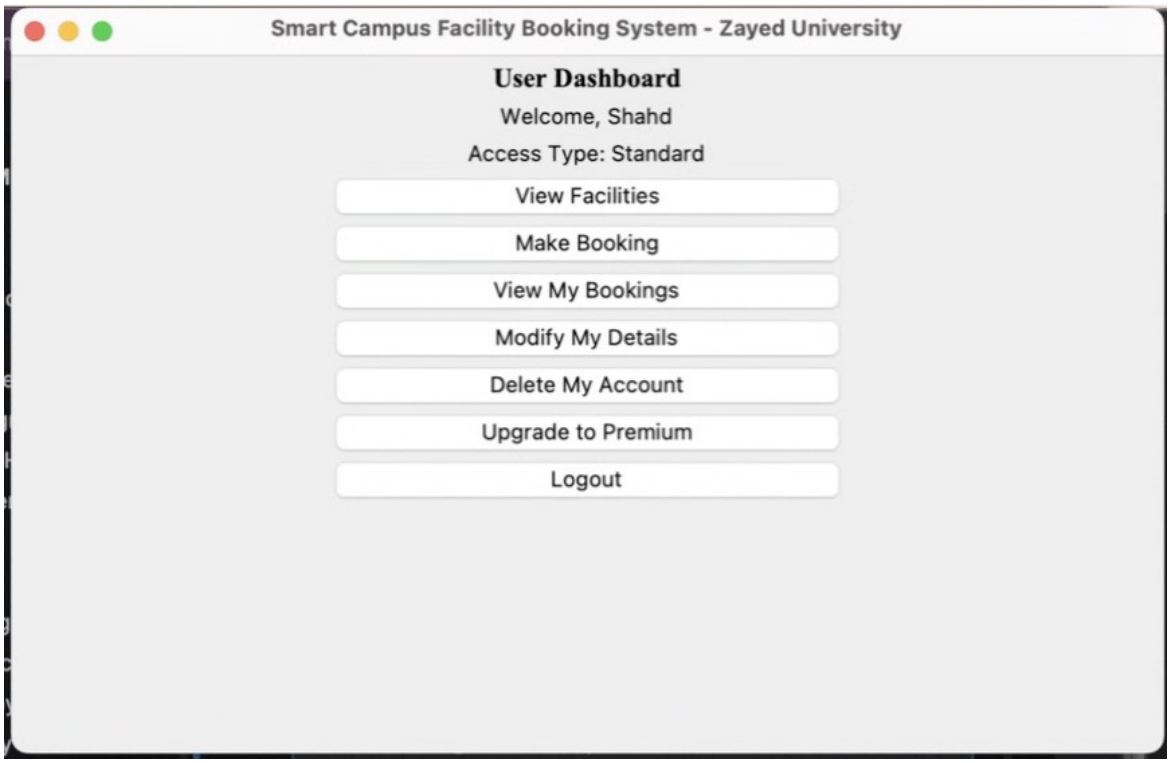


When we selected Create Account we entered the user details including name, email, and password. After clicking the create account button, the system displayed a message saying "Account created successfully." This test checks that the system correctly allows new users to register and stores their information properly.

4 - Login

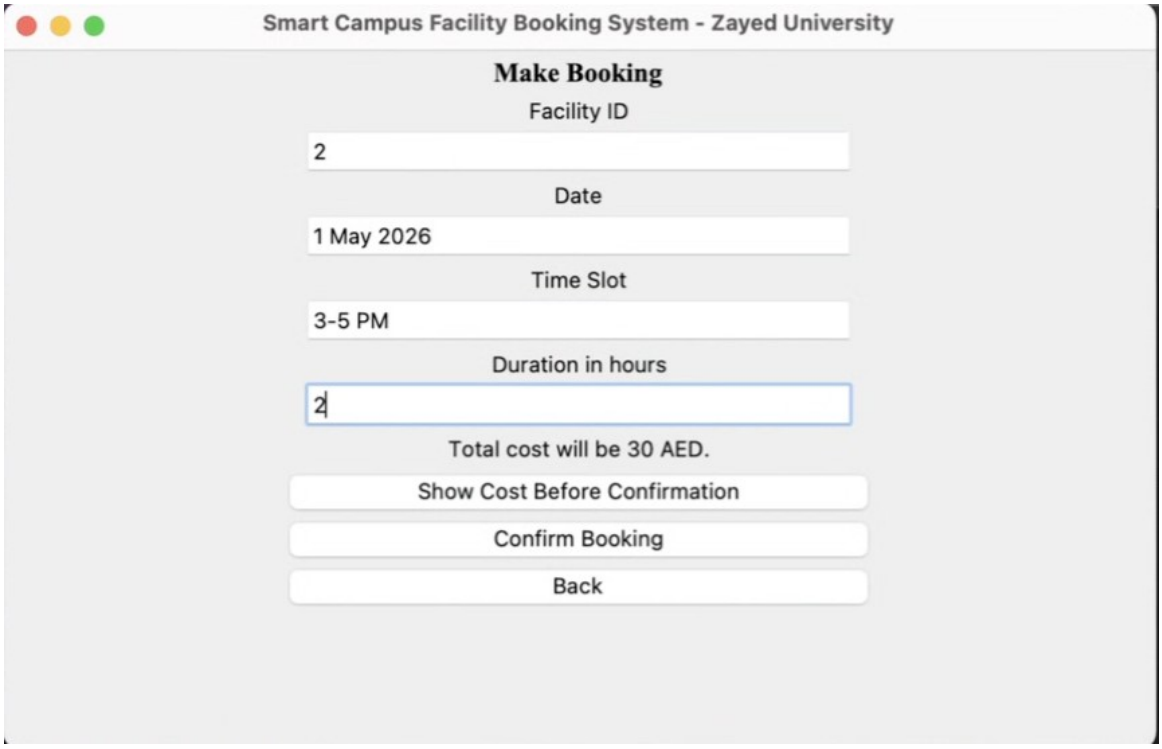


When we selected Login, we entered the correct email and password that were previously registered.

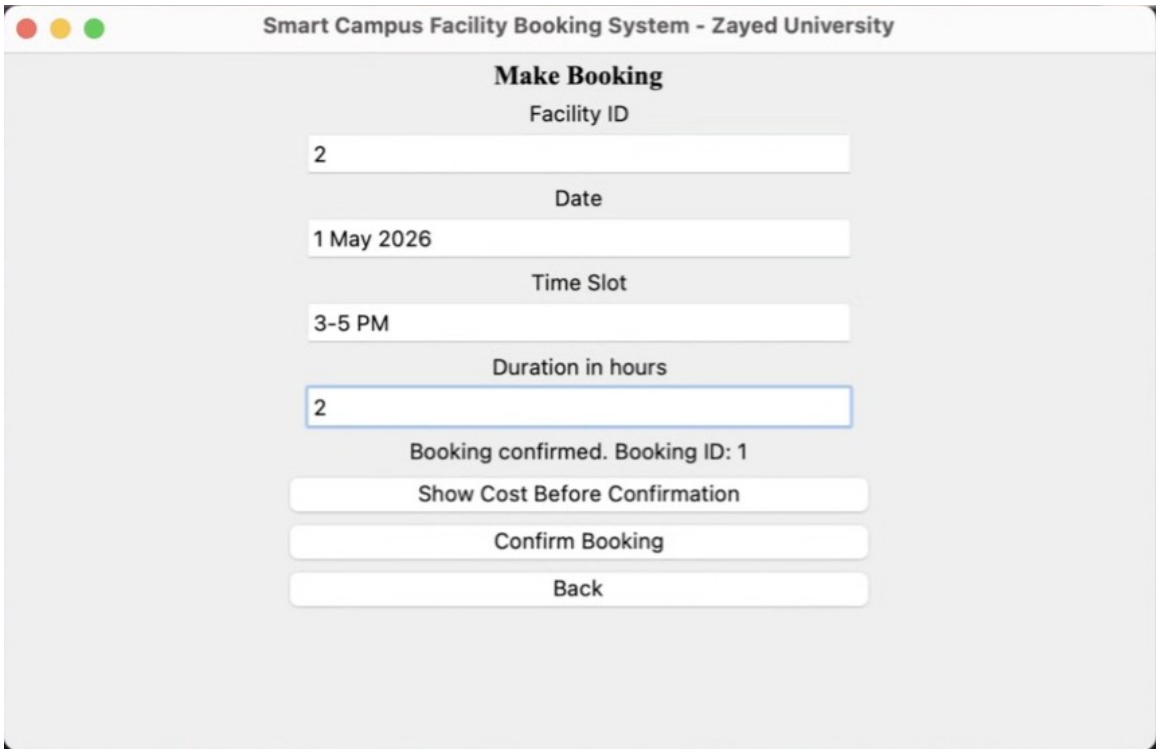


After clicking the login button the system successfully logged us in and redirected us to the user dashboard. This test checks that the system correctly verifies user information and allows access when the details are correct.

5 - Make Booking

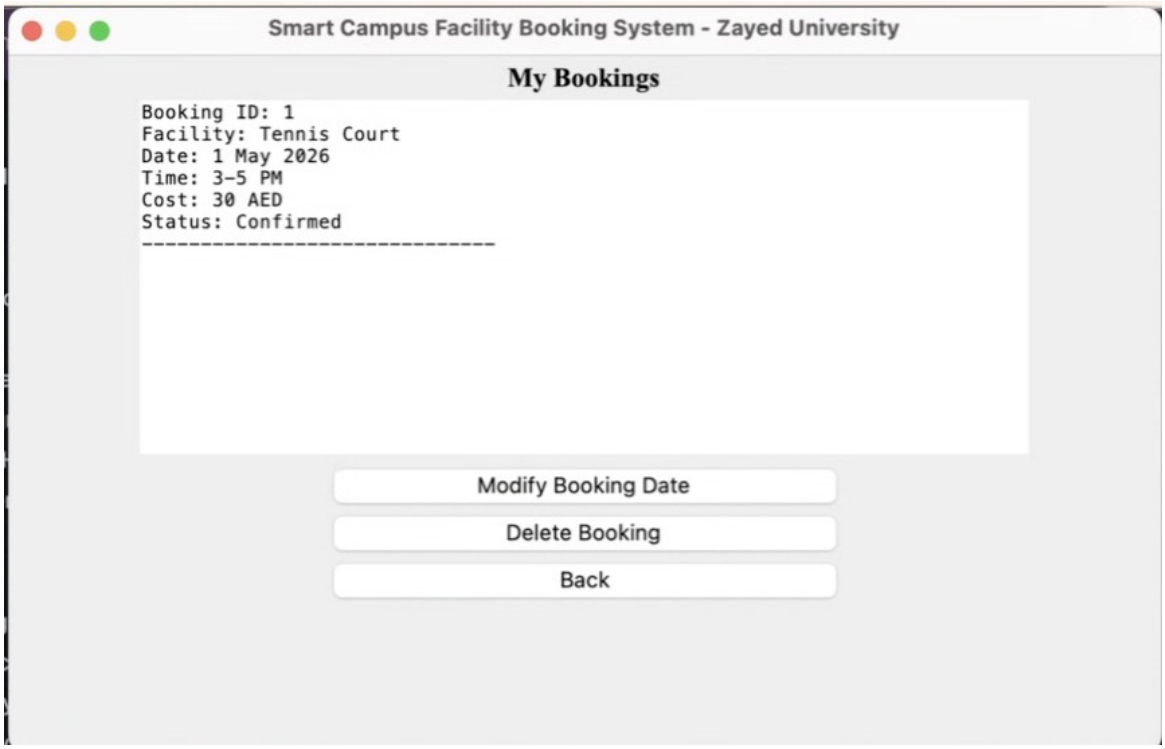


When we selected Make Booking, we entered the facility ID, date, time slot, and duration. The system automatically calculated and displayed the total cost based on the selected facility and duration.



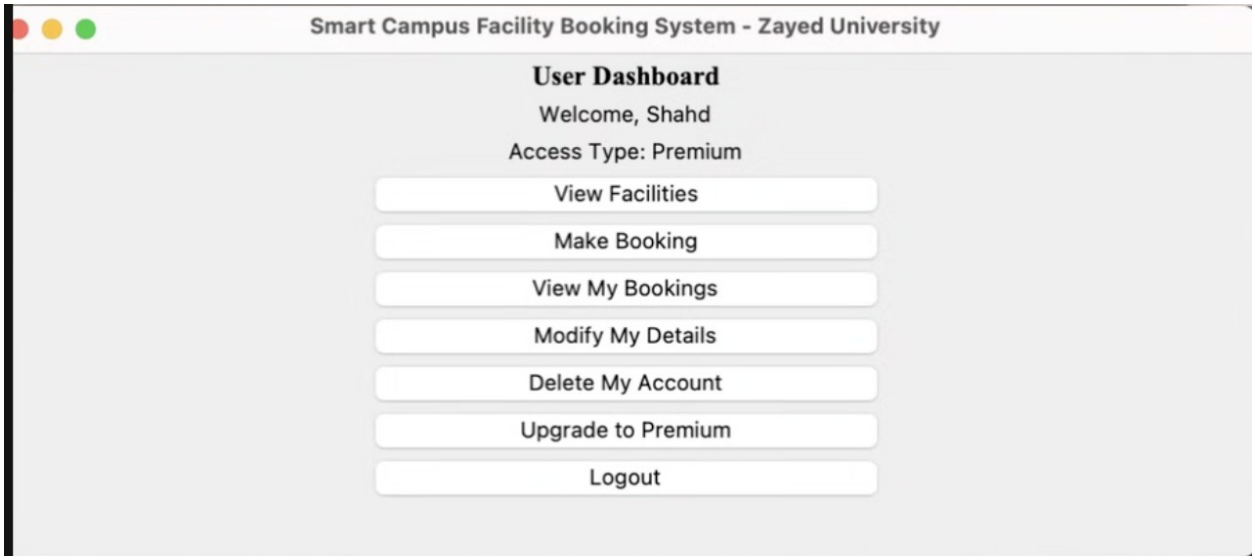
After confirming the booking, the system showed a confirmation message with the booking ID. This test checks that the system correctly creates a booking, calculates the cost, and confirms the reservation successfully.

6 - My Booking



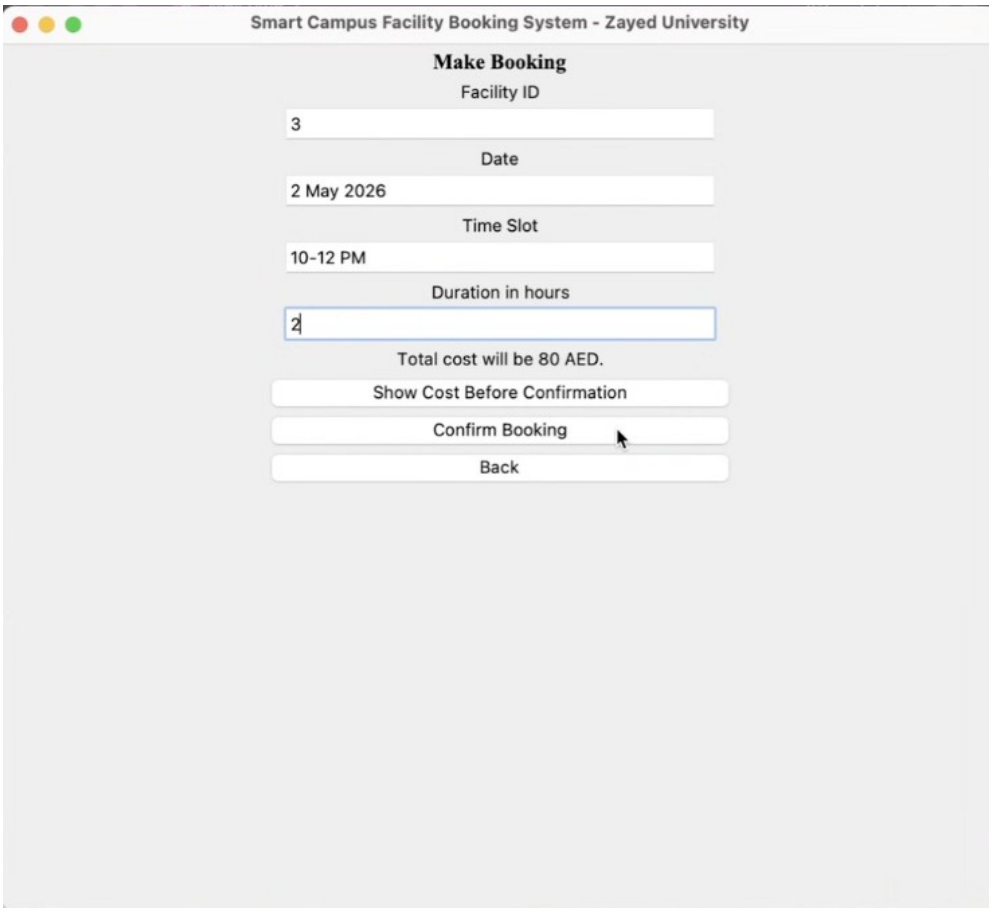
When we selected View My Bookings, the system displayed all the bookings that we made, including details such as booking ID, facility name, date, time, cost, and status. The system also provided options to modify the booking date or delete the booking. This test checks that the system correctly retrieves and displays user bookings and allows basic booking management actions.

7 - Premium

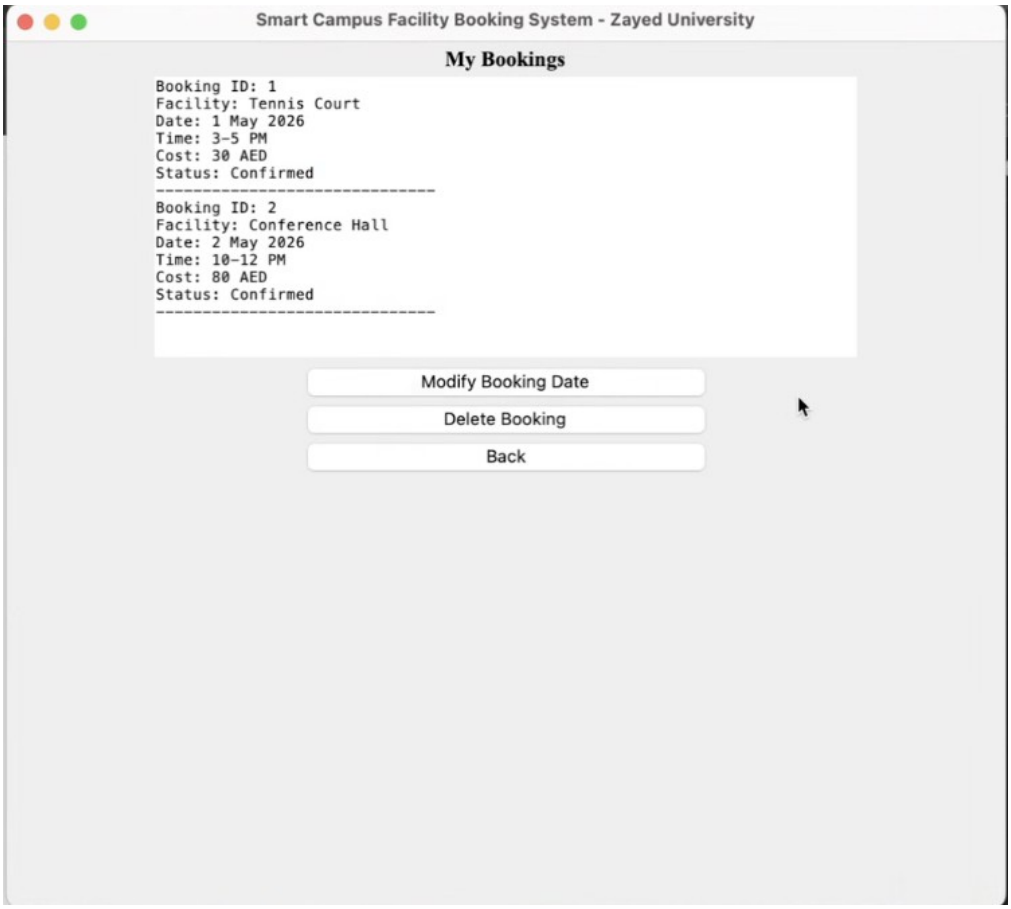


After logging in and upgrading the account, we accessed the user dashboard with premium access.

8 - Make Booking

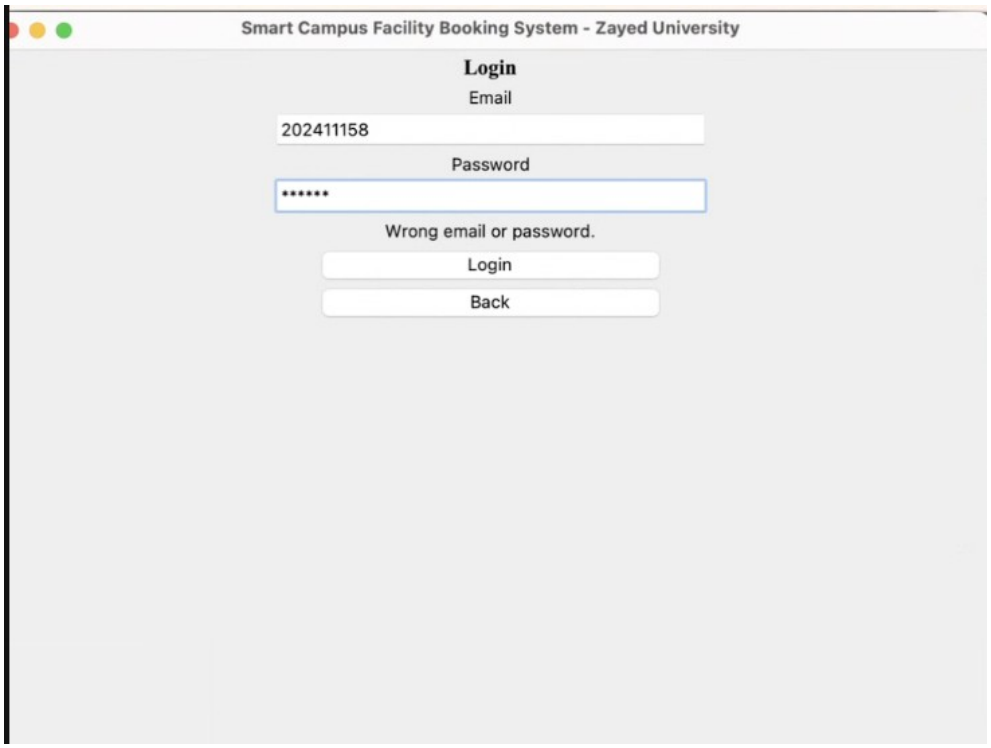


When we selected Make Booking, we entered a different facility ID along with the date, time slot, and duration. The system calculated and displayed the correct total cost based on the selected facility's price per hour. After confirming, the booking was successfully created and stored.



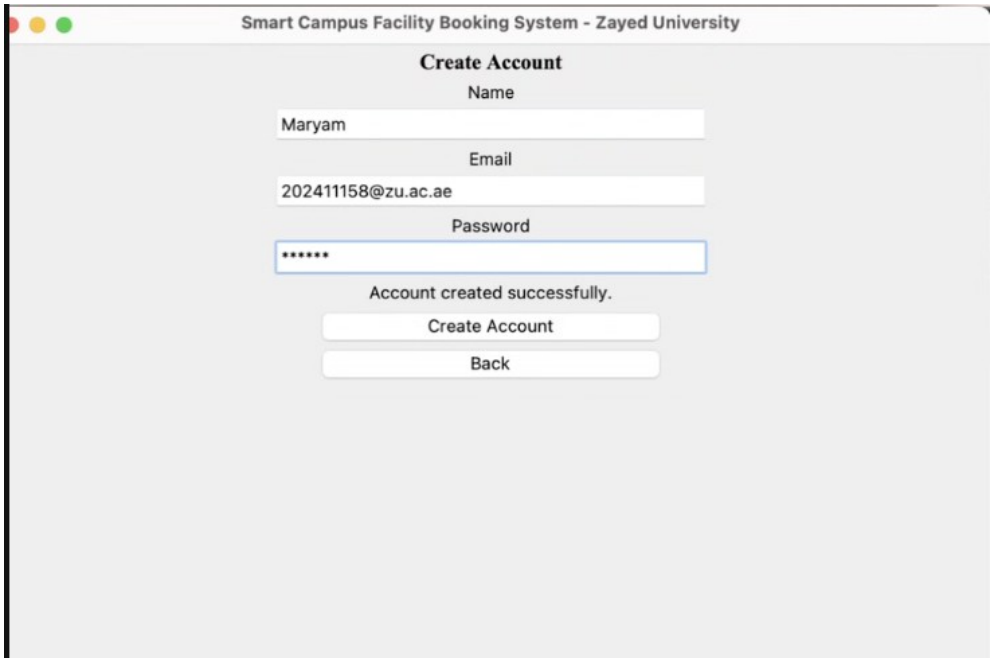
When we selected View My Bookings again, the system displayed multiple bookings that we created, showing all details such as booking ID, facility, date, time, cost, and status. This confirms that new bookings are saved correctly and added to the list. This test checks that the system can handle multiple bookings and display them accurately to the user.

9 - Login Unsaved Account

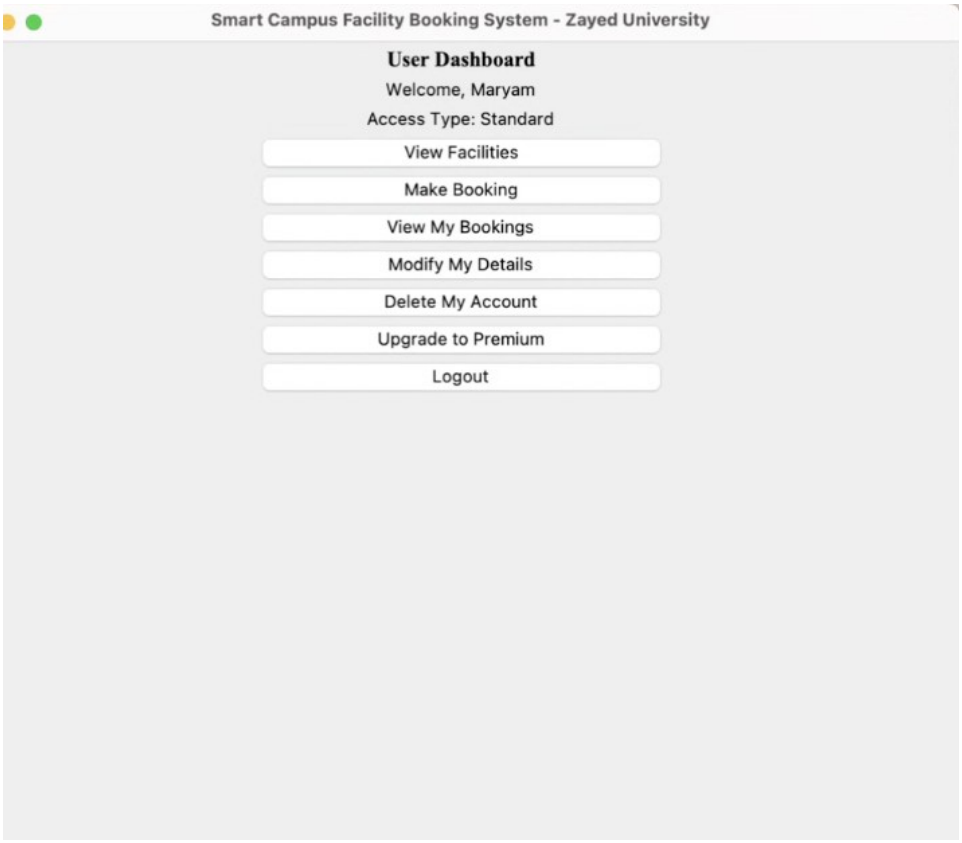


When we attempted to log in using account details that were not saved, the system displayed an error message indicating wrong email or password. This shows that the system does not allow access unless the account is properly saved. This test checks that the system validates login data correctly and prevents users from logging in with unsaved or invalid accounts.

10 - Multiple Accounts

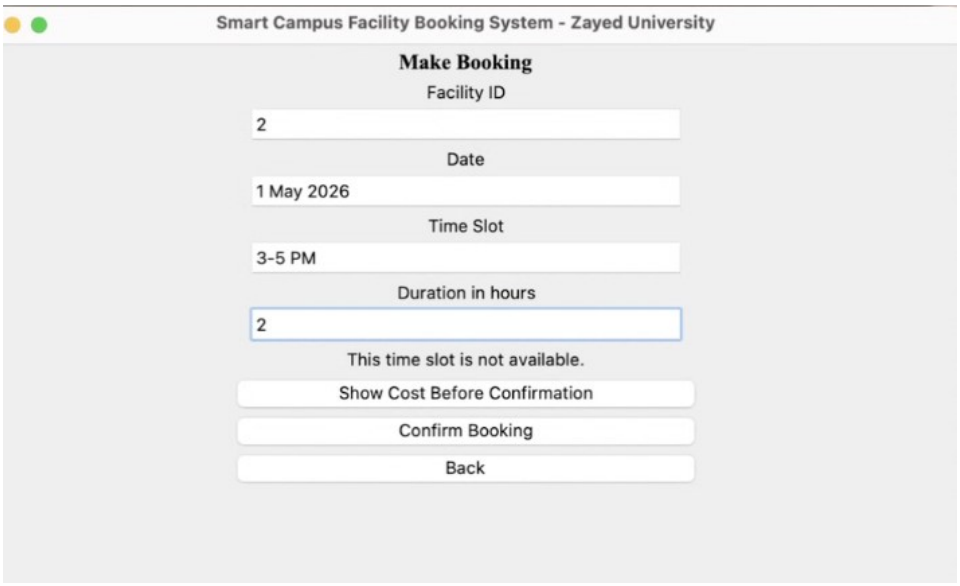


When we created another account using different user details, the system successfully registered the new user and displayed a confirmation message. This shows that the system supports multiple accounts and can store more than one user without conflicts. This test checks that multiple users can be created and managed correctly in the system.



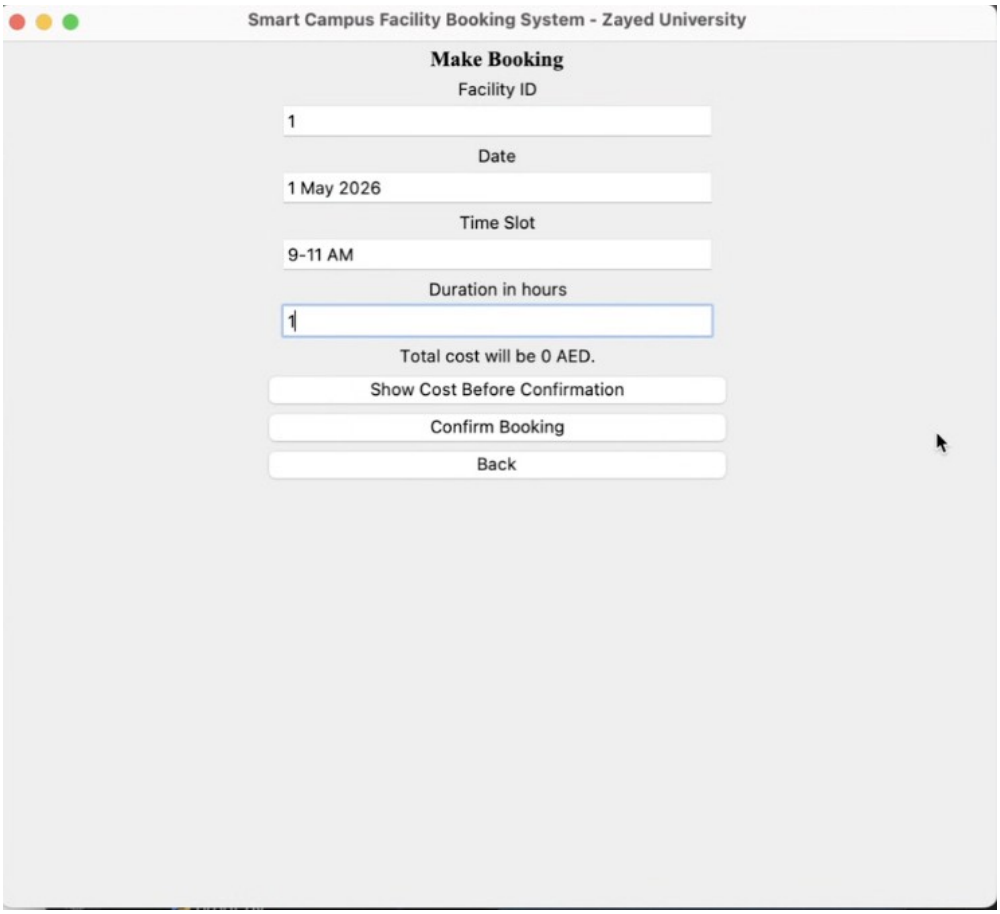
When we logged in using the newly created account, the system successfully redirected us to the user dashboard and displayed the correct user name and access type. This confirms that the account was created and stored correctly. This test checks that newly registered users can log in and access the system without any issues.

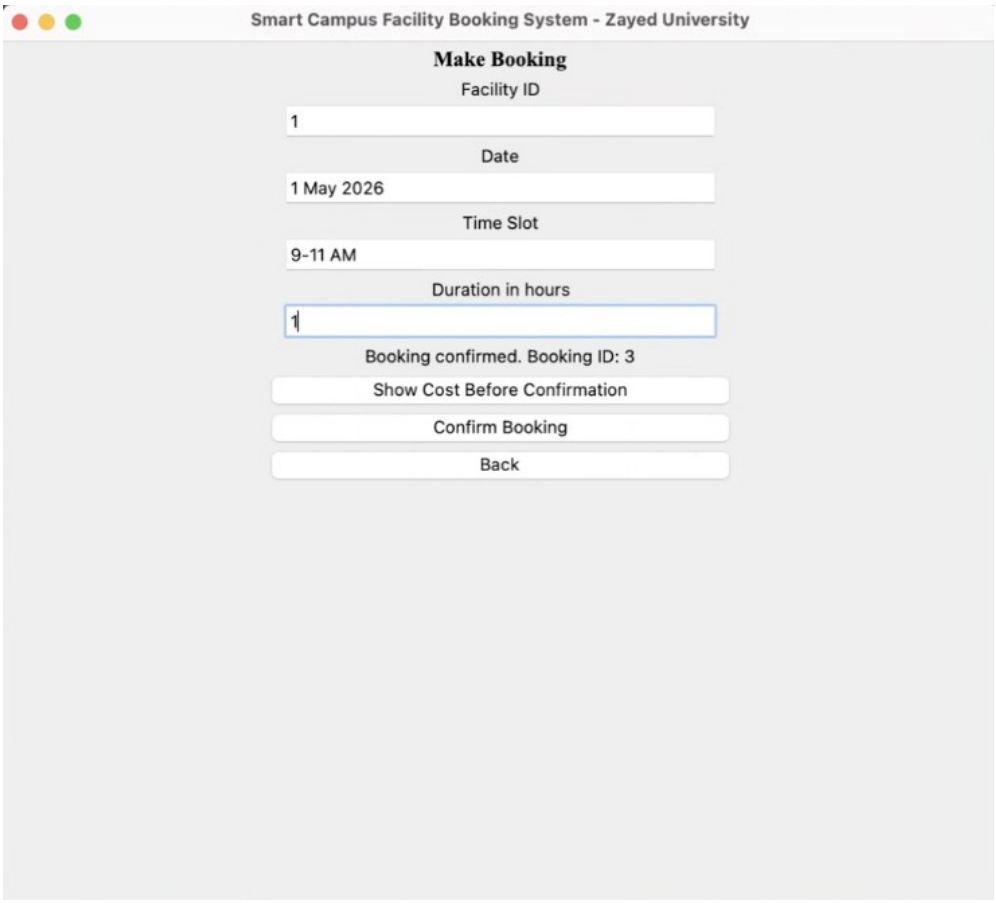
11 - Make Booking Used Time Slot



When we tried to make a booking using a time slot that was already booked, the system displayed a message indicating that the time slot is not available. This shows that the system correctly checks availability before confirming a booking. This test checks that double bookings are prevented and that the system maintains correct scheduling.

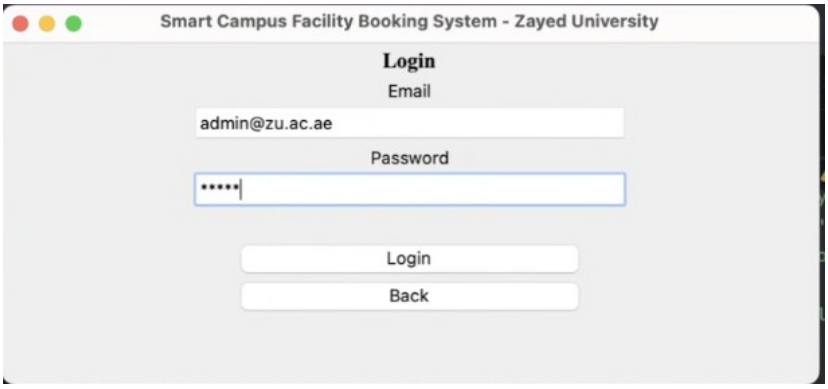
12 - Make Booking

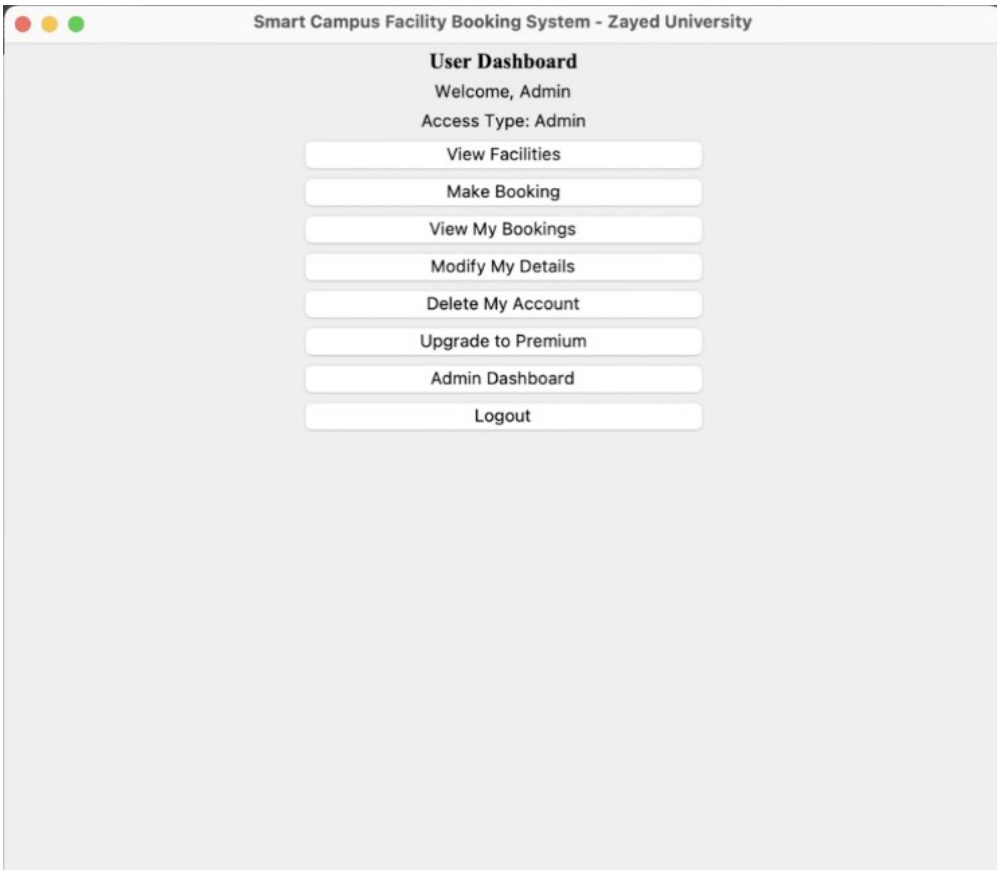




When we made a different booking using an available time slot, the system allowed us to proceed and successfully confirmed the booking with a booking ID. This shows that the system correctly processes valid bookings and updates the records. This test checks that users can book available slots without issues and receive confirmation.

13 - Admin Login



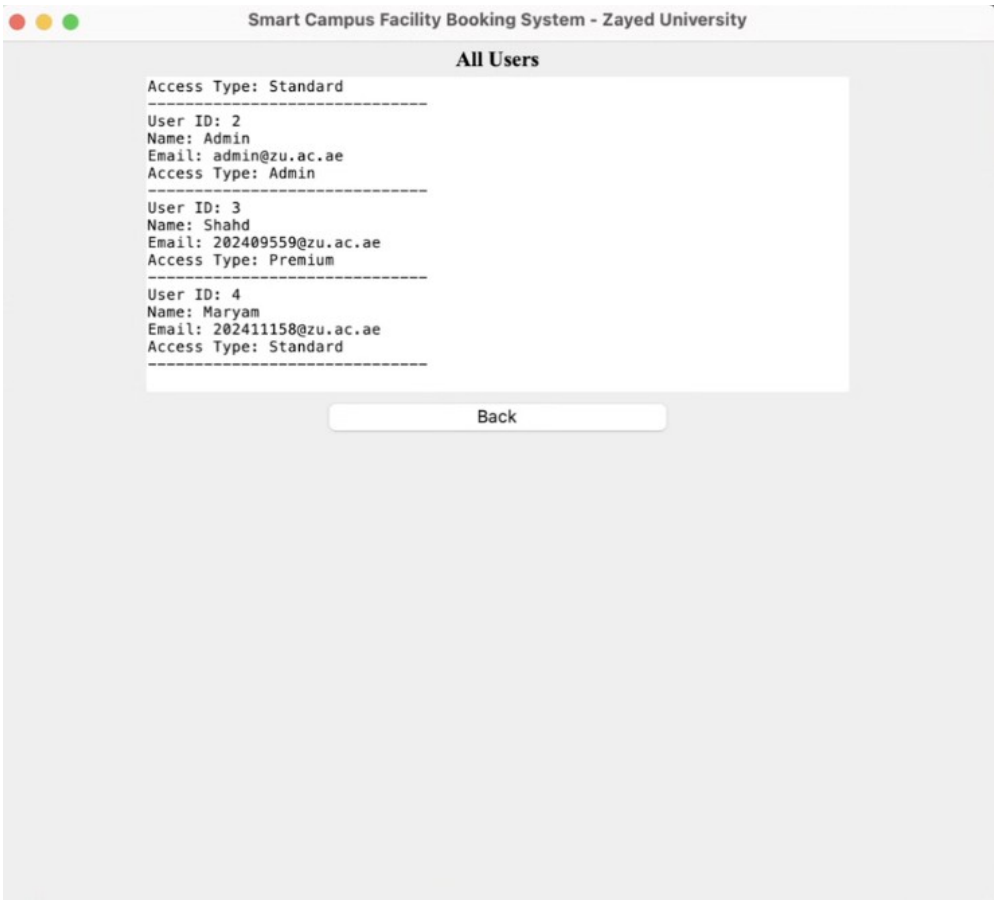


When we logged in as an admin, the system displayed the user dashboard with admin access type and additional options such as Admin Dashboard. This shows that the system correctly recognizes admin accounts and provides extra privileges compared to regular users. This test checks that role-based access control is working properly and that admin features are available when logging in with an admin account.

14 - Admin Dashboard

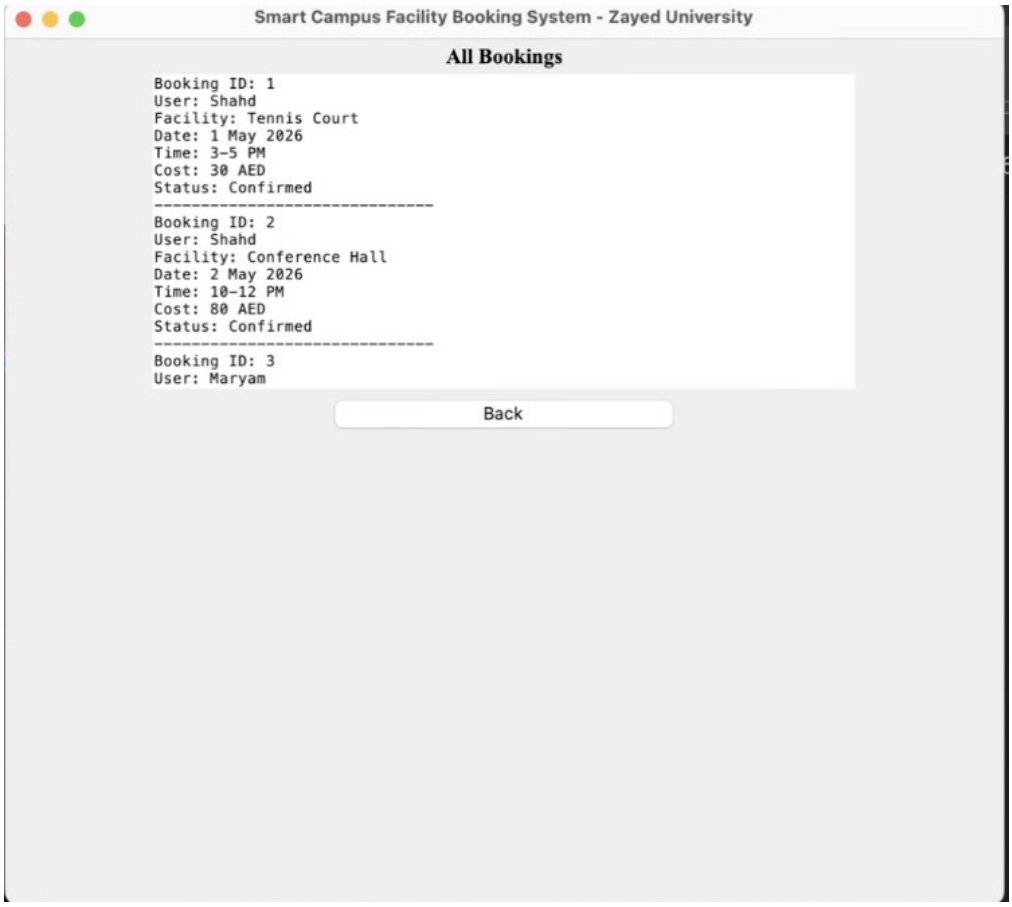
When we accessed the admin dashboard, the system displayed all admin specific options such as viewing all bookings, viewing all users, upgrading user access, updating facility availability, and viewing facility usage. This shows that the system provides full control and management features to the admin. This test checks that the admin dashboard works correctly and gives access to all administrative functionalities.

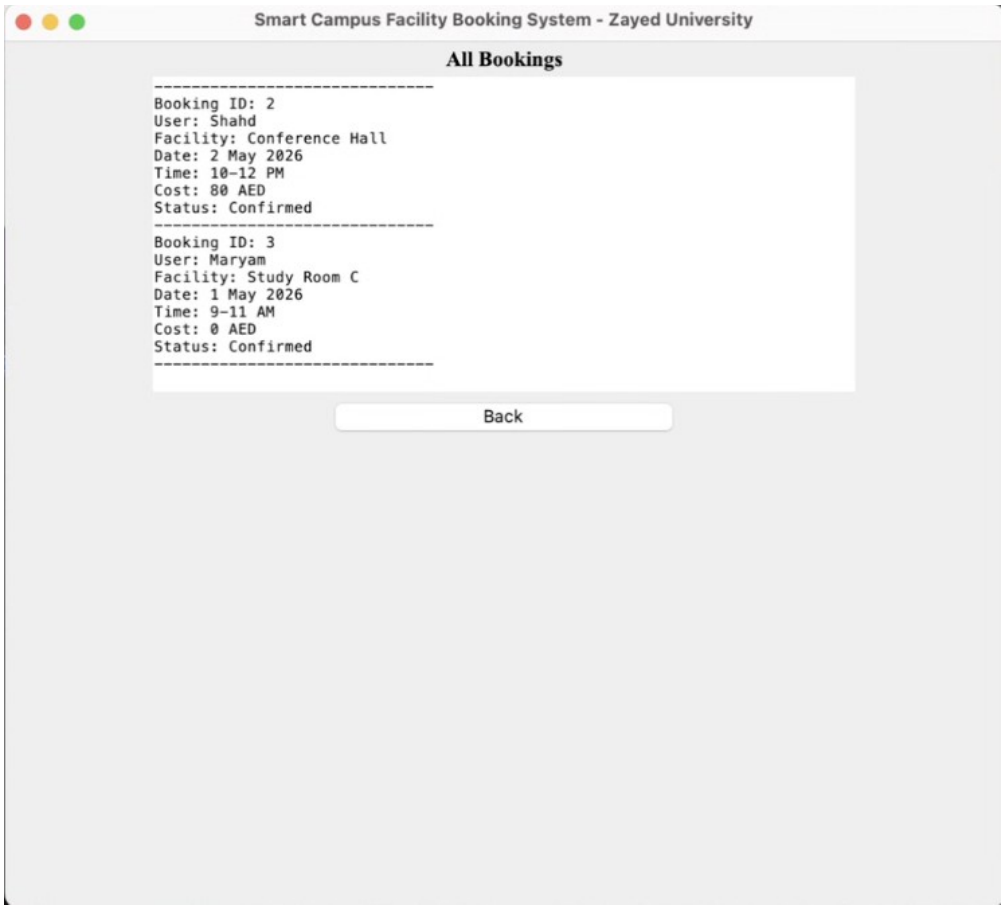
15 - View All Users



When we selected the option to view all users from the admin dashboard, the system displayed a list of all registered users along with their details such as user ID, name, email, and access type. This shows that the system correctly stores and retrieves user information for admin use. This test checks that the admin can view and manage all users in the system.

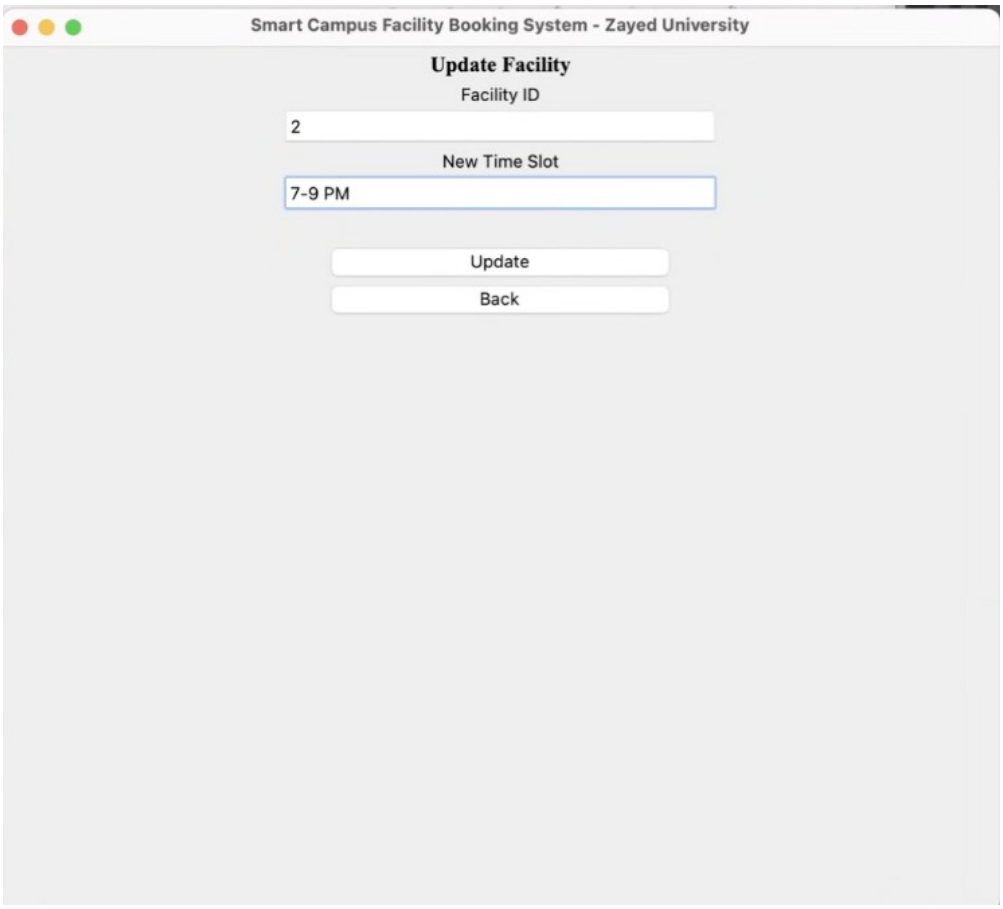
16 - View All Bookings

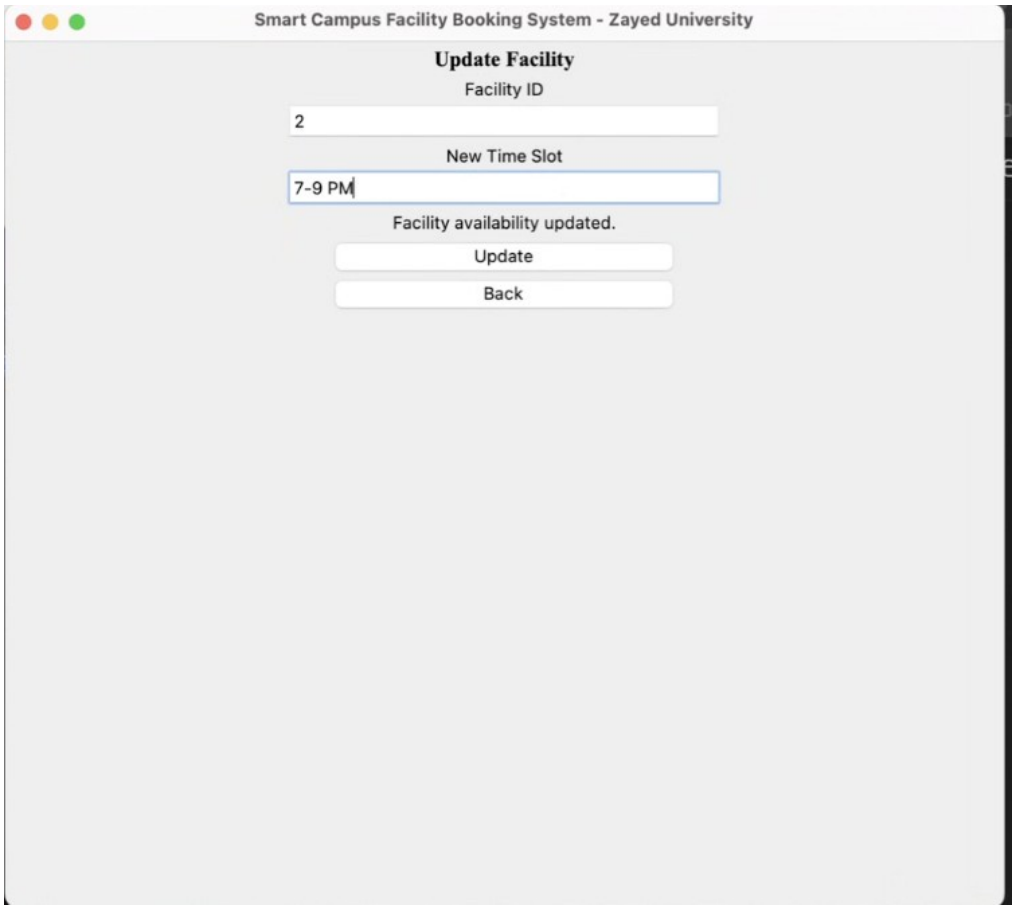




When we selected the option to view all users from the admin dashboard, the system displayed a list of all registered users along with their details such as user ID, name, email, and access type. This shows that the system correctly stores and retrieves user information for admin use. This test checks that the admin can view and manage all users in the system.

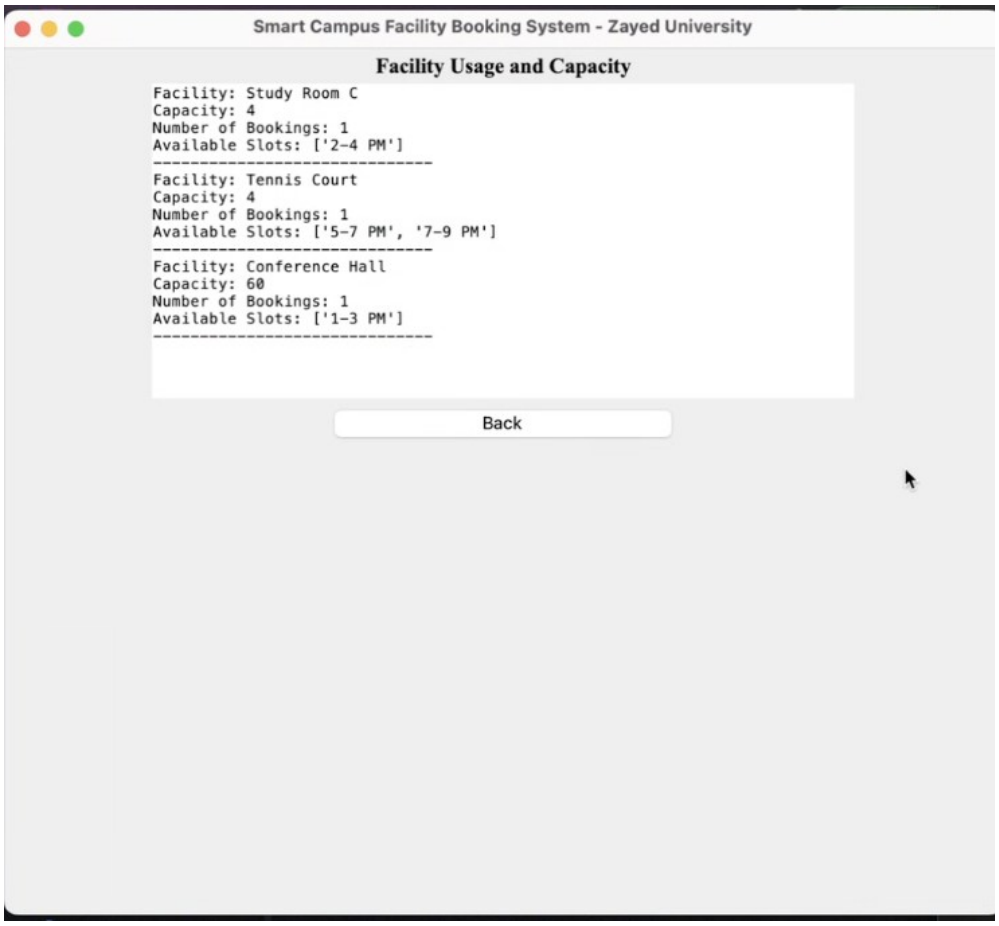
17 - Update Facility





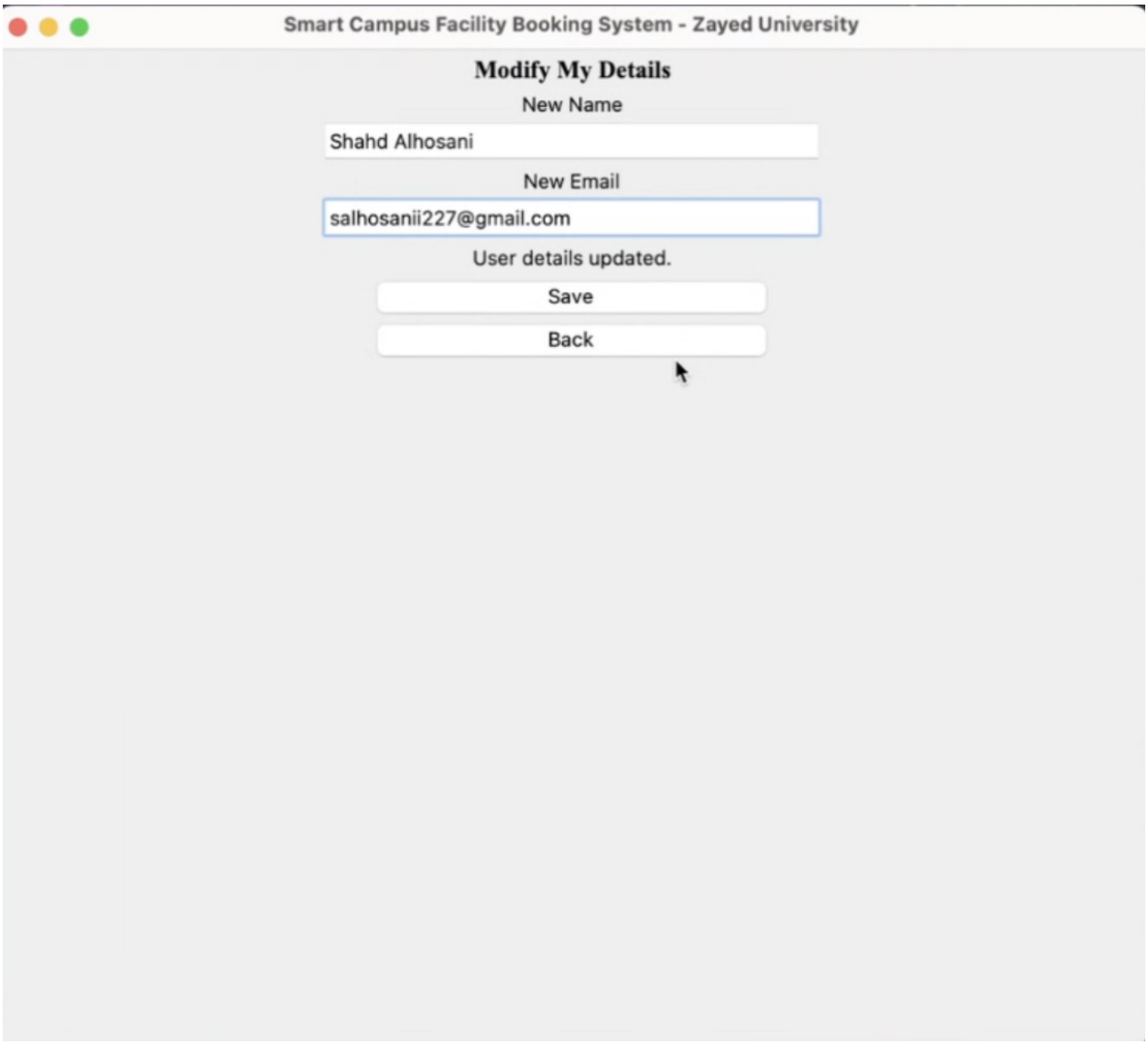
When we selected the option to update facility availability, the system allowed us to enter a new time slot for the selected facility and successfully updated it. This shows that the system enables the admin to manage and modify facility schedules. This test checks that facility availability can be updated correctly and reflected in the system.

18 - View Updated Facility Usage and Capacity



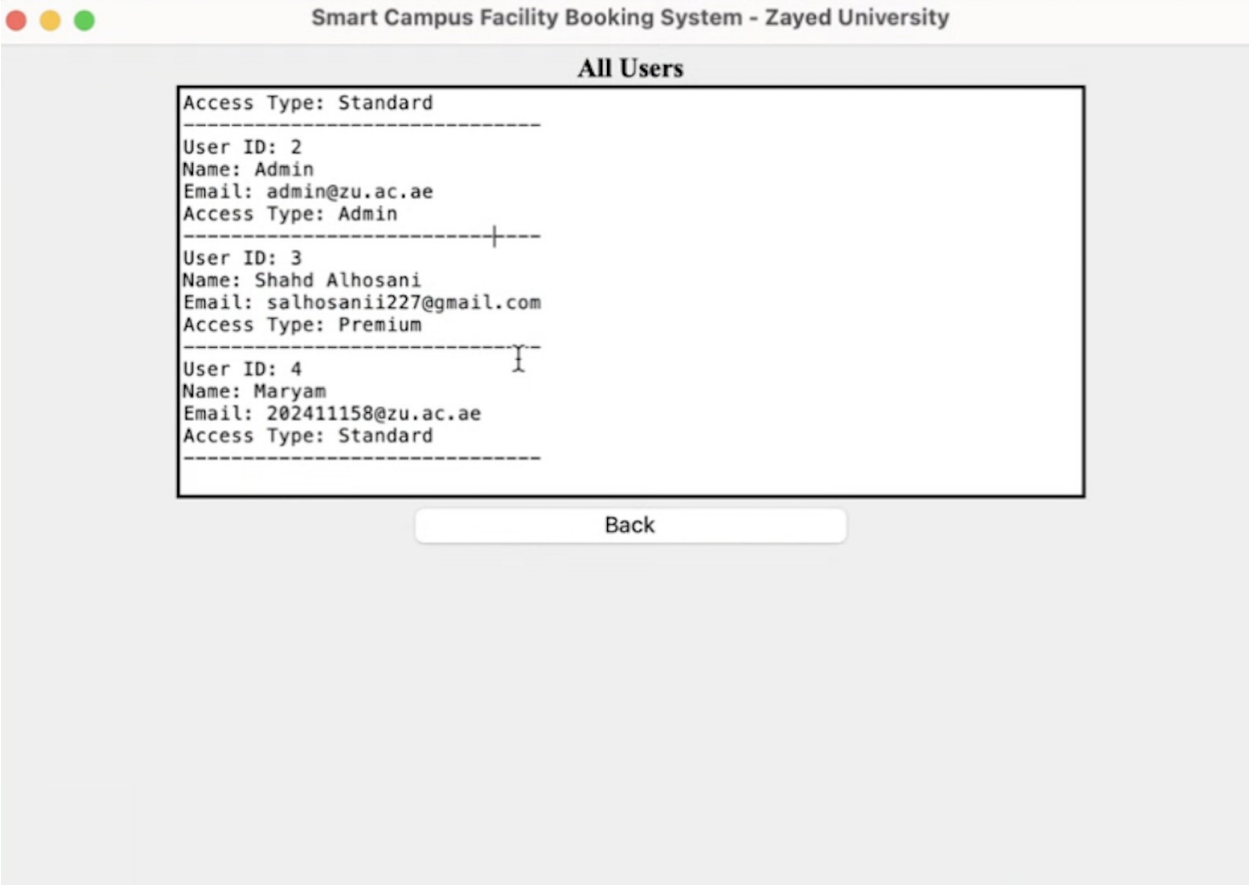
When we selected the option to view facility usage, the system displayed details for each facility including capacity, number of bookings, and available time slots. This shows that the system correctly tracks and updates facility usage after bookings and changes. This test checks that facility information is accurate and reflects current availability and usage.

19 - Modify My Details



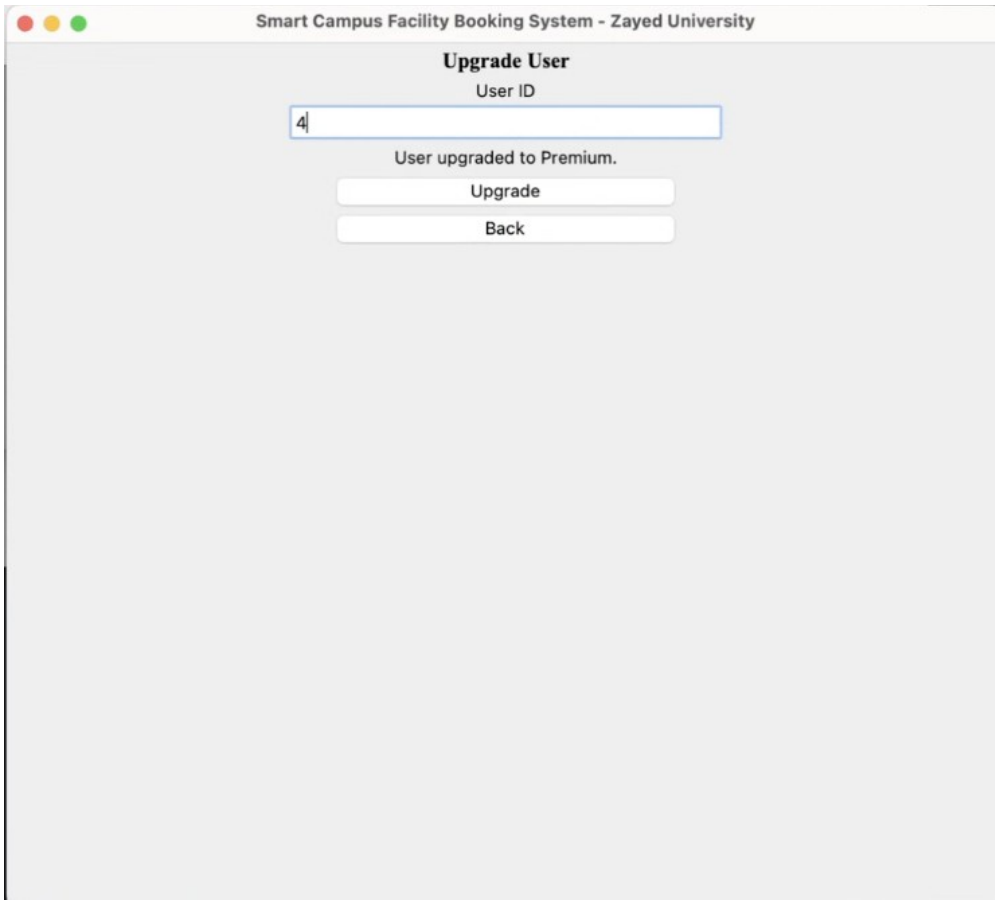
When we entered Shahd's account and modified the user details, the system allowed us to update the name and email successfully. This shows that the system supports editing user information and saving the updated details correctly. This test checks that users can modify their account information without issues.

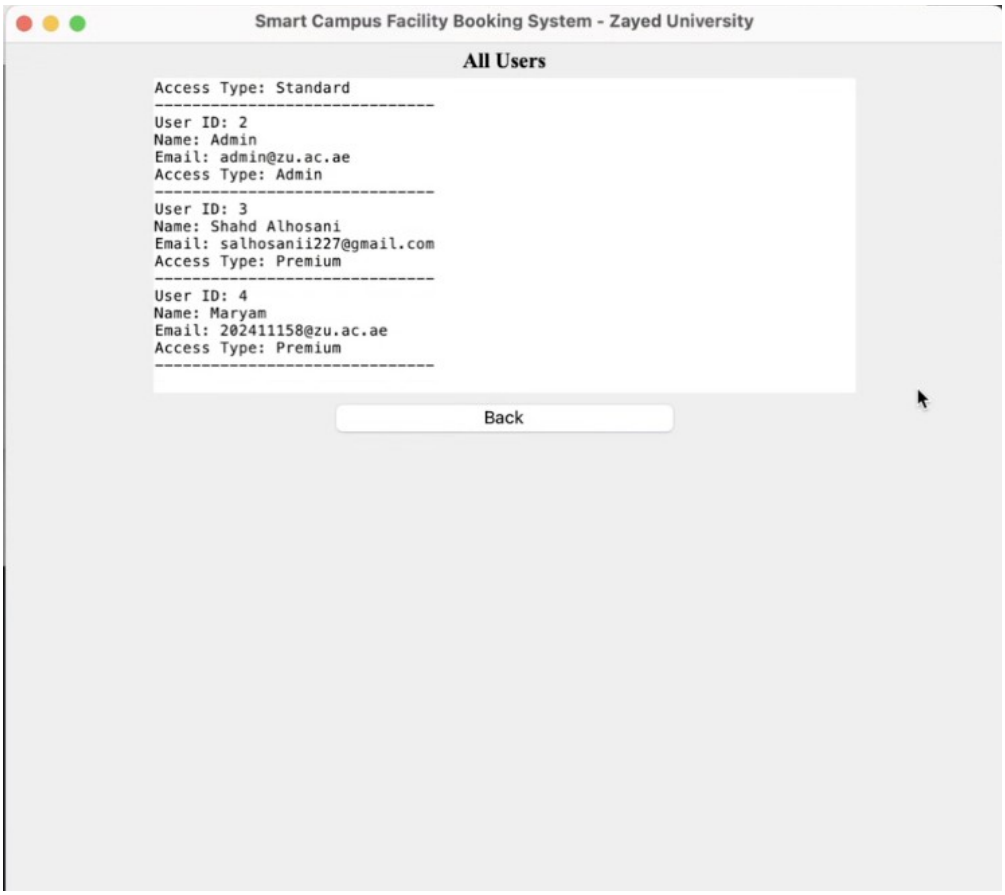
20 - Admin Updated Users



When we viewed all users from the admin dashboard, we saw that the updated user details were successfully reflected in the system. This shows that the modifications made to the user account were saved correctly and are visible to the admin. This test checks that user updates are stored properly and displayed accurately in the system.

21 - Admin Upgrade User Access





When we upgraded user 4 from the admin dashboard, the system successfully changed the user's access type to premium. After that, when we viewed all users, the updated access level was correctly shown. This shows that the system allows admins to upgrade user accounts and properly saves the changes. This test checks that user access levels can be updated and reflected accurately in the system.

4. Summary of learnings

In this project, we created a Smart Campus Facility Booking System using Python, Tkinter GUI, Object-Oriented Programming (OOP), and Pickle for data storage. The purpose of the system was to help students and staff book campus facilities such as study rooms sports courts and event halls in an organized and easy way. The project followed the assignment requirements and included both user and admin features.

We first started by designing a UML Class Diagram to plan the structure of the system. The main classes we created were User, Admin, Facility, Booking, and BookingSystem. The diagram also showed the relationships between the classes. For example, the Admin class inherited from the User class because admins are users with extra permissions. We also connected the classes using associations, such as users making bookings and facilities being linked to bookings.

After completing the UML diagram we implemented the Python code in PyCharm. Each class had its own attributes and methods based on its role in the system. The User class stored user information and allowed login and account upgrades. The Admin class allowed admins to view

bookings, update facility availability, and upgrade users to Premium access. The Facility class stored details such as facility type, capacity, available time slots, and booking price. The Booking class managed booking details, booking confirmation, cancellation, and total cost calculation. The BookingSystem class controlled all users, facilities, and bookings together.

We also created a full GUI using the Tkinter library. The system included several pages such as the main menu, login page creates account page, user dashboard, booking pages, modify booking page, delete booking page, modify user details page, and admin dashboard. The GUI made the system more user-friendly and easier to use.

The system also included important booking rules. Standard users could only reserve one facility type, while Premium users could reserve multiple facility types. Before confirming a booking, the system checked if the selected time slot was available. It also calculated the booking cost automatically depending on the facility price and booking duration.

To save data permanently, we used the Pickle library. This allowed user accounts, facilities, and bookings to be stored in binary files so the information would remain saved even after closing the program. We also added error handling to prevent problems such as empty fields, invalid IDs, duplicate emails, and incorrect inputs. Helpful messages were displayed to guide the user when errors happened.

Finally, we created and tested multiple test cases to make sure all features worked correctly. We tested account creation, login, making bookings, modifying bookings, deleting bookings, updating user details, upgrading users, and updating facility availability. Screenshots were included as evidence to show the successful working scenarios.

Overall, this project helped us improve our programming, GUI design, problem-solving, and teamwork skills while developing a complete and functional campus booking system.